

©2026 IEEE. Personal use of this material is permitted. Permission from IEEE must be obtained for all other uses, in any current or future media, including reprinting/republishing this material for advertising or promotional purposes, creating new collective works, for resale or redistribution to servers or lists, or reuse of any copyrighted component of this work in other works.

RL-Guided PAC-NMPC for Probabilistically-Safe Perception-Based Navigation in Unknown Environments

Adam Polevoy^{1,2}, Dillon Capalongo², Katherine Tang², Mark Gonzales², Marin Kobilarov², and Joseph Moore^{1,2}

Abstract—In this paper, we present an approach for combining stochastic nonlinear model predictive control (SNMPC) and reinforcement learning (RL) to enable probabilistically-safe perception-based navigation in unknown environments. Our method first uses RL to train probabilistic actor-critic and sensor prediction models. We then leverage these probabilistic models in a sampling-based SNMPC framework known as Probably Approximately Correct (PAC)-NMPC, which uses hard constraints to enforce finite-time statistical guarantees on the probability of collision and value function improvement. By ensuring that our finite-horizon SNMPC policies decrease the value function in expectation, we can approach the long-horizon performance of the RL approach while satisfying probabilistic safety constraints. Through simulation experiments, we show that our approach can improve the safety of perception-based RL navigation policies and scale to high dimensional systems with large sensor input spaces and complex nonlinear dynamics. We also demonstrate our approach through hardware experiments, showing improved performance for vision-based navigation with an agile fixed-wing aerial vehicle in unknown environments.

Index Terms—Aerial Systems; Perception and Autonomy, Optimization and Optimal Control, Robot Safety, Model Learning for Control

I. INTRODUCTION

Autonomous perception-based navigation in unknown environments remains a fundamental research challenge for agile, underactuated robots characterized by complex nonlinear dynamics and sizable state spaces. To navigate at-speed, these robots must predictively reason about their underlying dynamics while balancing computational efficiency, performance, and safety. During real-world deployment, not only must they consider the uncertainty in their own dynamics, but also the uncertainty associated with perception, including the uncertainty introduced by a limited field-of-view. Moreover, for many safety-critical applications, these algorithms must also be able to provide performance guarantees on metrics like collision-avoidance and task completion.

Nonlinear model predictive control (NMPC) is one approach for achieving perception-based navigation for a large class of robotic systems. Given a model of the system dynamics and an objective function, NMPC generates control inputs by repeatedly solving a finite-horizon optimization problem which can also be subject to nonlinear constraints. Robust and stochastic NMPC (RNMP, SNMPC) explicitly model uncertainty in the underlying dynamical system, and in some cases,



Fig. 1: Actor-Critic PAC-NMPC for navigation and obstacle avoidance on a fixed-wing with an on-board depth camera.

can provide safety guarantees (e.g., [1], [2], [3]). Often, such guarantees rely on restrictive assumptions about the structure of the dynamics and uncertainty. In addition, the computational burden of NMPC scales with the complexity of the system dynamics model. As a result, NMPC is often limited to short finite-horizons, resulting in policies with myopic behavior which can significantly limit long-range performance. In some cases, NMPC has been combined with sensor models for perception-based navigation and control (e.g., [4], [5]).

Reinforcement learning (RL) is another approach for achieving perception-based robot navigation for a general class of robotic systems. Modern, deep RL policies are often trained prior to controller deployment, thus eliminating the need for real-time optimization. Since the policies are often trained to minimize accumulated cost over a discounted infinite horizon, RL approaches typically show improved long-range performance. However, when these policies are only optimized prior to deployment, they can also exhibit degraded performance when operating outside of the training distribution. In general, RL approaches also lack a mechanism to explicitly enforce hard constraints, especially in out-of-distribution environments. Furthermore, because the data requirements of RL often necessitate training in simulation, there is frequently a distribution shift during sim-to-real transfer.

In this article, we present an approach that uses RL-trained networks within a sampling-based SNMPC framework to achieve probabilistically-safe perception-based navigation. In particular, we use a sampling-based SNMPC approach known as Probably Approximately Correct (PAC)-NMPC, which can provide statistical guarantees on cost and constraint satisfaction to ensure safety according to Def. III.6. Instead of training the RL networks jointly with the PAC-NMPC policy,

¹Johns Hopkins University Applied Physics Laboratory, Laurel, MD 20723, USA.

²Department of Mechanical Engineering, Johns Hopkins University, Baltimore, MD 21218, USA.

Email: Adam.Polevoy@jhuapl.edu, dcapalo1@jhu.edu, ktang21@jh.edu, mgonza60@jh.edu, marin@jhu.edu, jlmooore@jhu.edu

we train the RL networks independently and use Monte-Carlo (MC) dropout to approximate the policy and value function distributions. For high-dimensional sensor inputs, we also train a generative sensor network to predict future (unobserved) sensor returns. Given these learned probabilistic models, during deployment, we then warm-start our SNMPC approach using the RL policy distribution and sample terminal SNMPC costs from the value function distribution.

By combining SNMPC and RL in this way, our approach achieves three distinct advantages. First, it improves the safety of the RL policy according to Def. III.6 by using PAC-NMPC to provide statistical finite-time run-time guarantees on constraint satisfaction (e.g., local collision avoidance). Second, it also uses PAC-NMPC to provide statistical guarantees on value function improvement over the finite SNMPC horizon and dramatically improves the long-range optimality of the SNMPC policy. Third, by warm-starting the SNMPC policy with the learned actor and using a predictive sensor model, the SNMPC algorithm can be decoupled from the RL policy during training and thus remove the computational burden of an inner NMPC optimization loop. We demonstrate the performance improvements of our approach through simulation and hardware experiments and evaluate our method on the particularly challenging problem of vision-based collision-free navigation with an agile fixed-wing aerial robot.

This manuscript substantially extends the work in [6], which introduced an approach for combining PAC-NMPC with a learned stochastic value function. In this paper, we significantly advance this approach to scale to higher state and measurement spaces and three-dimensional environments, and to provide finite-time guarantees in unknown environments. Our key contributions are as follows:

- 1) A constrained PAC-based framework for combining SNMPC with RL to enable both long-horizon planning and enforcement of statistical safety guarantees.
- 2) A learned model to predict future sensor measurements, for which closed form dynamics are unavailable in unknown environments
- 3) An RL-based SNMPC policy warm-starting method for scaling to higher dimensional state measurement spaces.
- 4) A hardware evaluation of our approach on a fixed-wing aerial vehicle with an on-board depth camera, demonstrating real-time control of robotic systems with local perception and complex nonlinear dynamics.

II. RELATED WORK

A number of approaches have been proposed to improve the safety of RL policies or improve the long-range performance of receding-horizon NMPC. In this section, we provide a brief overview of prior related research on safe RL, learned NMPC warm-starting, NMPC with learned waypoints, and hybrid RL-NMPC methods. We also review prior research on perception-based fixed-wing navigation, since we use this unique planning and control problem to evaluate our approach.

A. Safe RL

Safe RL methods aim to improve the safety of RL policies by training and/or deploying learned policies that minimize ex-

pected future costs while adhering to safety constraints. These problems are often posed as Constrained Markov Decision Processes (CMDP). In [7], the authors provide an overview of recent safe RL methods and in [8], the authors provide a more general review of learned safe control under uncertainty.

Safe policy optimization methods rely on data-driven approaches to learn safe policies. Constrained policy gradient methods [9] modify the policy update in an attempt to maintain satisfaction of expected future constraints. Other approaches use Lagrangian relaxation to approximate the problem as unconstrained optimization [10]. However, once outside of the training distribution, these approaches can no longer guarantee constraint satisfaction.

Control theory based methods introduce explicit models to regulate inputs to ensure safety, thus providing stronger assurances. Many approaches require knowledge of a Lyapunov function in advance to guarantee safety [11]. Likewise, control barrier functions (CBFs) are often utilized to guarantee safety [12]. While the CBFs can be learned from data, they assume that safety can be represented completely as a function of the state, which may not be possible when only partial observations of the environment are available. Safety layer methods take possibly unsafe inputs and project them to inputs that satisfy constraints. OptLayer follows this methodology by augmenting the policy with a constrained optimization layer [13]. While this framework is similar to our approach, it only guarantees one-step safety and does not consider dynamics or perception uncertainty.

Instead of trying to enforce safety constraints, other approaches aim to provide a statistical performance guarantee for the learned policy. PAC-NMPC takes inspiration from PAC Robust Policy Search [14], [15], which optimizes a PAC guarantee on the expected cost of the policy. However, these guarantees do not necessarily hold outside of the training distribution. PAC-Bayes control [16] provides both PAC guarantees on policy performance and generalization guarantees to novel environments by assuming bounded divergence between the training and testing distributions.

B. Learned NMPC Warm-start

Compared to RL, NMPC can more naturally enforce run-time safety constraints. However, the global optimality of these algorithms is typically limited by computational resources and susceptibility to local minima. To improve this long-range performance, researchers have investigated methods that use machine learning to warm-start the NMPC optimization. Many approaches build data sets of optimal trajectories offline and train models to predict policies or trajectories [17], [18]. In [19], the authors used trajectory data to learn warm-starts for sequential convex programming with obstacle avoidance constraints. Researchers have also learned value functions to select from a history of stored controller data while providing stability guarantees [20].

Other approaches have focused on initializing not only the control input sequence, but also the associated constraints. In [21], the authors learn a warm start for the active set, and in [22], authors scaled learned primal active set initialization to higher dimensional systems. More recently, authors developed

a set of constraint-informed merit functions for training, thus making feasibility the target of the warm start [23].

Researchers have also begun to explore using more advanced generative models for learned warm-starting. For instance, in [24], researchers used a transformer-based architecture to learn a terminal cost for long horizon guidance. In [25], the authors use Motion Transformer to generate multi-modal warm-starts for escaping local minima in fast-changing traffic scenarios. TransformerMPC [26] improved optimization solve time by training a transformer to select only active constraints in the MPC problem. In [27], authors proposed a diffusion-based approach conditioned on object-centric perception data.

While learned warm-starting has been applied to many diverse and challenging problems such as bipedal locomotion [28], self-driving cars [25], and free-flying space robotic platforms [29], few of these approaches have considered planning with local perception in unknown environments.

C. NMPC with Learned Waypoints

In addition to learned warm-starting, researchers have utilized learned waypoints to improve the long-range navigation capabilities of finite-horizon NMPC. In some cases, researchers have used reinforcement learning to generate these waypoints for navigation in unknown environments [30], quadrotor navigation in environments with dead-end corridors [31], and navigation in dynamic environments with other agents [32]. In other cases, supervised learning has been combined with offline perception-based kinodynamic planning to enable drone racing [33] and navigation in real-world cluttered environments [34]. These approaches often assume a separation of the high-level and low-level planning problems and are often unable to reason about the cases where planning and perception are highly coupled during dynamic maneuvers.

D. NMPC & RL Hybrids

Researchers have also explored NMPC-RL hybrid methods to overcome the limitations inherent to either NMPC or RL alone in scenarios where high and low-level planning are not easily decoupled. In these architectures, the RL policy often improves the long-horizon behavior, while the NMPC approach improves runtime policy performance, and in some cases, improves convergence during RL training. Many of these approaches utilize NMPC controllers as the RL policy itself. PETS [35] and POLO [36] use NMPC policies to learn a probabilistic dynamics model and a probabilistic value function, respectively. MQP [37] integrates the action-value function into an information-theoretic NMPC objective. Related approaches, such as DMPC [38], which incorporates the value function into the stage cost, and CACTO [39], which leverages NMPC warm-starting in the training loop, demonstrate further convergence improvements. While several of these approaches can be applied to systems with stochastic dynamics [35], [36], [37] and in some cases a stochastic value function [36], they do not provide safety guarantees.

Safe RL-MPC [40] provides safety guarantees of form in Def. III.6 by learning the parameters of a robust linear MPC controller. However, because nonlinearities must

Approach	Stochastic MPC	Stochastic Value Func.	Safety Guarantees	Perception (max dim)	Hardware Demo
DMPC[38], CACTO[39]	✗	✗	✗	✗	✗
PETS[35], MQP[37]	✓	✗	✗	✗	✗
DiffStack[44]	✗	✗	✗	✓(5×a)*	✗
AC-MPC[46]	✗	✗	✗	✓(12×2)	✓
POLO[36]	✓	✓	✗	✗	✗
LOOP[42], TD-MPC[43]	✓	✗	✗	✓(16, 84×84×9)	✗
Safe RL-MPC [40], PSF[41]	✓	✗	✓	✗	✗
AC-PAC-NMPC (Ours)	✓	✓	✓	✓(16×12)	✓

* DiffStack uses a 4-D observation and ego indicator for each agent in the scene.

TABLE I: Comparison of MPC & RL hybrid approaches.

be modeled as disturbances, generated control policies are likely to be overly conservative. Another method, predictive safety filters [41] modifies unsafe learned policy inputs using robust constraint-tightening NMPC, which can also lead to conservative policies [2]. Neither approach has been applied to systems with perception.

LOOP [42] learns both an observation dynamics model and a value function, and TD-MPC [43] learns an observation dynamics model in latent space to improve sample efficiency. However, neither approach provides safety guarantees. DiffStack [44] and Actor Critic MPC [45] embed differentiable MPC controllers directly into a learned policy. While both of these approaches can accommodate perception inputs, they do not consider stochastic dynamics or provide safety guarantees.

Table I provides a comparison of the aforementioned RL-NMPC hybrid approaches. Not only does our approach (Actor-Critic PAC-NMPC) reason about both stochastic dynamics and a stochastic value function, but it also provides probabilistic safety guarantees and can accommodate sizable perception input spaces. In addition, we evaluate our approach through hardware experiments and demonstrate the method’s applicability to a challenging planning and control task.

E. Vision-based Agile Fixed-Wing Flight

To evaluate our approach on a highly dynamic system with local perception and complex dynamics, we consider vision-based navigation through an unknown obstacle field with an agile fixed-wing aerial vehicle. Here, we briefly review some of the prior approaches explored to address this problem.

Trajectory libraries have been one method for planning and control of fixed-wing robots [47], [48], [49]. Because these libraries are constructed offline, many of the computational challenges associated with generating trajectories for aerial robots, especially across the full flight envelope can be avoided. These approaches have also been more tightly integrated with motion planning approaches to improve performance and enable long-range planning [50], [51], [52].

Online trajectory optimization and NMPC are another class of methods for controlling fixed-wing robots. At low angles-of-attack, a number of researchers have leveraged differentially flat representations to achieve real-time trajectory optimization

[53], [54], [55]. To achieve NMPC over a larger flight envelope, researchers have used nonlinear optimization approaches like sequential quadratic programming (SQP) [56], [57], [58], MPPI [59], or PAC-NMPC [60]. Researchers have also explored differentially flat representations for tail-sitter UAVs that can support aerobatic flight [61].

Fewer approaches have explored perception-based navigation with fixed-wing UAVs, since these approaches often require leveraging light-weight stereo depth cameras with limited fields-of-view which are rigidly mounted to the aircraft body and thus tightly coupled to aircraft motion. Vision-based fixed-wing navigation and obstacle avoidance has been demonstrated with trajectory libraries for low angle-of-attack reactive obstacle avoidance [49], [62] and using NMPC for agile flight in urban environments [4], [57]. In [16], researchers used reinforcement learning to generate robust policies for vision-based navigation with a tail-sitter UAV.

In this paper, we demonstrate through both simulation and hardware experiments that our hybrid RL-MPC approach can generate probabilistically safe policies for agile fixed-wing flight and enable probabilistically-safe perception-based obstacle avoidance and navigation.

III. PROBLEM FORMULATION

In this paper, we design a controller capable of navigating a robotic system through an unknown, unstructured environment using on-board perception. The robot must reach the goal, $\mathbf{x}_G$, while avoiding collisions with partially observed obstacles. We represent the evolution of the robot's state over time as a stochastic dynamical system.

Definition III.1 (Stochastic Dynamics). *Given states $\mathbf{x}_{t+1}$, $\mathbf{x}_t \in \mathcal{X}$ and input $\mathbf{u}_t \in \mathcal{U}$, the stochastic dynamics is given as $p(\mathbf{x}_{t+1} \mid \mathbf{x}_t, \mathbf{u}_t)$, where t is the current time index.*

Throughout this paper, we assume that $p(\mathbf{x}_{t+1} \mid \mathbf{x}_t, \mathbf{u}_t)$ depends continuously on $\mathbf{x}_t$ and $\mathbf{u}_t$.

We assume that the robot has a perception sensor that provides partial observations of obstacles in the environment.

Definition III.2 (Sensor Measurement). *Let $O \in \mathcal{O}$ define the obstacles in the environment. Then, the sensor measurement is $\mathbf{y}_t = \mathbf{h}(\mathbf{x}_t, O)$ where $\mathbf{h} : \mathcal{X} \times \mathcal{O} \mapsto \mathcal{Y}$ and $\mathbf{y}_t \in \mathcal{Y}$.*

Together, the state and sensor measurement represent for the robot's observation of the world.

Definition III.3 (Observation). *The observation is given as $\mathbf{z}_t = (\mathbf{x}_t, \mathbf{y}_t)$ where $\mathbf{z}_t \in \mathcal{X} \times \mathcal{Y}$.*

We define a stage cost and constraint to define the desired navigation behavior of the robot. Specifically, the cost encourages progress towards the goal, $\mathbf{x}_G$, while discouraging undesired behaviors. The constraint, dependent on a history of sensor measurements, defines unsafe states and inputs.

Definition III.4 (Stage Cost). *The stage cost, $\ell(\mathbf{x}_t, \mathbf{u}_t) \succ 0$, is a continuous function given as $\ell : \mathcal{X} \times \mathcal{U} \mapsto \mathbb{R}$.*

Definition III.5 (Stage Constraint). *The stage constraint, $c(\mathbf{x}_t, \mathbf{u}_t, \mathbf{y}_{(t-N_y):t})$, is a continuous function given as $c :$*

$\mathcal{X} \times \mathcal{U} \times \mathcal{Y}^{N_y} \mapsto \mathbb{R}$ where $c(\mathbf{x}_t, \mathbf{u}_t, \mathbf{y}_{(t-N_y):t}) \leq 0$ indicates constraint satisfaction.

We also define a probabilistic safety guarantee as follows:

Definition III.6 (Probabilistic Safety Guarantee). *Let $\mathcal{S} \subset \mathcal{X}$ be a safe-set. A probabilistic safety guarantee implies*

$$\mathbb{P}(\mathbb{P}(\mathbf{x}_{t+i} \in \mathcal{S}, \forall i \in \{0, \dots, N_T\} \mid \mathbf{x}_t) \geq 1 - \beta_S) \geq 1 - \delta_S$$

where β_S, δ_S are small positive values, and N_T is the horizon length.

For Def. III.6, $\delta_S = 0$ corresponds to Safety Level II in [8].

IV. BACKGROUND

Here we review PAC-NMPC [60], a receding-horizon sampling-based SNMPC algorithm, and actor-critic RL, which comprise the main components of our approach.

A. PAC-NMPC

PAC-NMPC is a sampling-based SNMPC approach that optimizes and provides probabilistic guarantees on both the expected cost and probability of constraint violation. It provides these guarantees in the form of PAC bounds calculated from sampled policies. Unlike many NMPC approaches, PAC-NMPC optimizes a distribution over feedback policies rather than an open-loop sequence of control actions.

Consider a state and input trajectory, $\tau = (\mathbf{x}_t, \mathbf{u}_t, \mathbf{x}_{t+1}, \mathbf{u}_{t+1}, \dots, \mathbf{x}_{t+N_T})$ where N_T is the trajectory horizon and $\tau \in \mathcal{T} \subset \mathcal{X}^{N_T+1} \times \mathcal{U}^{N_T}$. PAC-NMPC defines a trajectory cost and trajectory constraint violation indicator.

Definition IV.1 (Trajectory Cost). *The trajectory cost is given as $J : \mathcal{T} \mapsto \mathbb{R}_{\geq 0}$ such that*

$$J(\tau) = \sum_{i=t}^{t+N_T-1} \ell(\mathbf{x}_i, \mathbf{u}_i) + \ell_f(\mathbf{x}_{t+N_T}), \quad (1)$$

where $\ell_f(\mathbf{x}_{t+N_T})$ is the terminal cost.

Definition IV.2 (Trajectory Constraint Violation Indicator). *The trajectory constraint violation indicator is given as $C : \mathcal{T} \mapsto \{0, 1\}$ such that*

$$C(\tau) = \bigvee_{i=t}^{t+N_T-1} (c(\mathbf{x}_i, \mathbf{u}_i, \mathbf{y}_{(t-N_y):t}) > 0) \quad (2)$$

where $C(\tau) = 1$ indicates constraint violation.

PAC-NMPC optimizes local feedback policies, $\mathbf{u}_t = \pi_t(\mathbf{x}_t, \xi)$, which are parameterized by open-loop nominal input trajectories, $\xi = (\mathbf{u}_t^d, \dots, \mathbf{u}_{t+N_T-1}^d)$. To generate these policies, it uses the time-varying linear quadratic regulator (TVLQR) [63]. The nominal input trajectories are rolled out into state space from an initial state using a nominal, deterministic dynamics model, $\mathbf{x}_{t+1}^d = f(\mathbf{x}_t^d, \mathbf{u}_t^d)$, resulting in nominal state and input trajectories, τ^d . Time-varying feedback gains, $\mathbf{K}_t$, are computed from τ^d using TVLQR. The resulting feedback policy is $\mathbf{u}_t = \mathbf{K}_t(\mathbf{x}_t^d - \mathbf{x}_t) + \mathbf{u}_t^d$.

To optimize these feedback policies, PAC-NMPC defines a Gaussian surrogate exploration distribution, $\mathcal{N}(\xi \mid \mu, \Sigma)$, over the nominal input trajectory. The decision variables

of the optimization problem are the mean and variance of the surrogate distribution, $\boldsymbol{\nu} = (\boldsymbol{\mu}, \text{diag}(\boldsymbol{\Sigma}))$. This surrogate distribution is iteratively optimized to minimize the upper PAC bounds, $\mathcal{J}_\alpha^+(\boldsymbol{\nu})$ and $\mathcal{C}_\alpha^+(\boldsymbol{\nu})$, over the expected trajectory cost and probability of constraint violation:

$$\begin{aligned} \hat{\boldsymbol{\nu}}_{i+1}^* &= \arg \min_{\boldsymbol{\nu}} \min_{\alpha} (\mathcal{J}_\alpha^+(\boldsymbol{\nu}) + \gamma \mathcal{C}_\alpha^+(\boldsymbol{\nu})) \\ \text{s.t. } \quad &\mathbb{P}(\mathbb{E}[J(\boldsymbol{\tau})] \leq \mathcal{J}_\alpha^+(\boldsymbol{\nu})) \geq 1 - \delta \\ &\mathbb{P}(\mathbb{E}[C(\boldsymbol{\tau})] \leq \mathcal{C}_\alpha^+(\boldsymbol{\nu})) \geq 1 - \delta \end{aligned} \quad (3)$$

where δ is a user-defined confidence parameter. The resulting optimized surrogate distribution provides probabilistic guarantees, $\mathcal{J}_\alpha^+(\boldsymbol{\nu})$ and $\mathcal{C}_\alpha^+(\boldsymbol{\nu})$. Since $C(\boldsymbol{\tau})$ is a binary indicator, $\mathbb{E}[C(\boldsymbol{\tau})]$ represents the probability of constraint violation, resulting in a probabilistic safety guarantee of the same form as Definition III.6. The controller executes the feedback policy constructed from the maximum likelihood estimate, $\boldsymbol{\mu}$.

The PAC bounds take the form

$$\mathcal{J}_\alpha^+(\boldsymbol{\nu}) = \hat{\mathcal{J}}_\alpha(\boldsymbol{\nu}) + \alpha d(\boldsymbol{\nu}) + \Phi_\alpha \quad (4)$$

where $\hat{\mathcal{J}}_\alpha$ is a robust estimator of the expected cost, d is a distance term from prior sampled distributions, Φ_α is a concentration-of-measure term, and α is an annealing and regularizing coefficient. These bounds can be calculated from a finite number of policy samples.

Given L prior surrogate distributions, $\boldsymbol{\nu}_0, \dots, \boldsymbol{\nu}_{L-1}$, and M iid samples from each, $(\boldsymbol{\tau}_{i0}, \boldsymbol{\xi}_{i0}), \dots, (\boldsymbol{\tau}_{iM-1}, \boldsymbol{\xi}_{iM-1})$, the robust estimate of the expectation is given as

$$\hat{\mathcal{J}}_\alpha(\boldsymbol{\nu}) = \frac{1}{\alpha LM} \sum_{i=0}^{L-1} \sum_{j=0}^{M-1} \psi(\alpha \ell_{ij}) \quad (5)$$

$$\psi(x) = \log \left(1 + x + \frac{1}{2}x^2 \right) \quad (6)$$

$$\ell_{ij} = J(\boldsymbol{\tau}_{ij}) \frac{p(\boldsymbol{\xi}_{ij}|\boldsymbol{\nu})}{p(\boldsymbol{\xi}_{ij}|\boldsymbol{\nu}_i)}. \quad (7)$$

The distance term is given as

$$d(\boldsymbol{\nu}) = \frac{1}{2L} \sum_{i=0}^{L-1} b_i^2 e^{D_2(p(\cdot|\boldsymbol{\nu})||p(\cdot|\boldsymbol{\nu}_i))} \quad (8)$$

$$0 \leq J(\boldsymbol{\tau}_{ij}) \leq b_i \forall j = 0, \dots, M-1 \quad (9)$$

where D_2 is the Renyi divergence.

The concentration-of-measure term is given as

$$\Phi_\alpha = \frac{1}{\alpha LM} \log \frac{1}{\delta}. \quad (10)$$

These bounds are local to each finite-horizon policy and conditional on current observations. Thus, they provide finite-time guarantees and do not guarantee global safety for closed-loop execution of the controller over the infinite horizon.

B. Actor-Critic RL

Reinforcement learning algorithms usually involve the estimation of the value function, which represents the minimum expected accumulation of future costs. In our case, the value function can be interpreted as the optimal cost-to-go from the current state, $\mathbf{x}_t$, to the goal state, $\mathbf{x}_G$. For the following

definitions, we let $\mathbf{x}_t$ denote the full state of the environment, rather than the state of the robot alone.

Definition IV.3 (Optimal Value Function). *Given some stage cost $g(\mathbf{x}_t, \mathbf{u}_t)$ and stochastic dynamics $p(\mathbf{x}_{t+1} | \mathbf{x}_t, \mathbf{u}_t)$, the optimal value function is given as $V : \mathcal{X} \mapsto \mathbb{R}$ such that*

$$V(\mathbf{x}_t) = \min_{\mathbf{u}_t \in \mathcal{U}} \left[g(\mathbf{x}_t, \mathbf{u}_t) + \beta \mathbb{E}[V(\mathbf{x}_{t+1}) | \mathbf{x}_t, \mathbf{u}_t] \right] \quad (11)$$

where $\beta \in (0, 1)$ is a discount factor.

Definition IV.4 (Optimal Policy). *The optimal policy is given as $\pi^* : \mathcal{X} \mapsto \mathcal{U}$ such that*

$$\pi^*(\mathbf{x}_t) = \arg \min_{\mathbf{u}_t \in \mathcal{U}} \left[g(\mathbf{x}_t, \mathbf{u}_t) + \beta \mathbb{E}[V(\mathbf{x}_{t+1}) | \mathbf{x}_t, \mathbf{u}_t] \right]. \quad (12)$$

Actor-critic methods often use temporal difference learning to jointly learn approximations of the optimal policy and optimal value function [64]. For most state-of-the-art off-policy methods, which benefit from improved sample efficiency, the critic, $Q^\psi(\mathbf{x}_t, \mathbf{u}_t)$, parameterized by ψ , approximates the optimal action-value function, $Q^*(\mathbf{x}_t, \mathbf{u}_t) = g(\mathbf{x}_t, \mathbf{u}_t) + \beta \mathbb{E}[V(\mathbf{x}_{t+1}) | \mathbf{x}_t, \mathbf{u}_t]$. The actor, $\pi^\phi(\mathbf{x}_t)$, parameterized by ϕ , approximates the optimal policy, $\pi^*(\mathbf{x}_t)$. The approximation of the optimal value function can be obtained by composing the actor and critic, $V^{\phi\psi}(\mathbf{x}_t) = Q^\psi(\mathbf{x}_t, \pi^\phi(\mathbf{x}_t))$. Popular actor-critic methods for continuous state and action spaces include PPO [65], SAC [66], and TD3 [67].

The optimal value function and policy depend on the full environment state, which includes information beyond the robot state, such as the obstacles, O . In our case, since the robot can only partially observe the world, the actor and critic must rely on the robot's observation, $\mathbf{z}_t$. Throughout this work, we interchangeably write $V^{\phi\psi}(\mathbf{z}_t)$ and $V^{\phi\psi}(\mathbf{x}_t, \mathbf{y}_t)$ where appropriate, and likewise for π^ϕ and Q^ψ .

V. ACTOR-CRITIC PAC-NMPC (AC-PAC-NMPC)

We now describe our approach for combining PAC-NMPC with actor-critic RL to enable improved long-range performance while providing probabilistic safety guarantees. Although the learned actor and critic do not provide safety guarantees themselves, they can safely be incorporated into PAC-NMPC to guide the optimization.

Our approach utilizes three learned models. We train an actor, $\pi^\phi(\mathbf{z}_t)$, and critic, $Q^\psi(\mathbf{z}_t, \mathbf{u}_t)$, using simulated dynamics and sensor models. Additionally, we train a sensor prediction model, $\hat{\mathbf{y}}_{t+k} = \mathbf{h}^\eta(\mathbf{x}_t, \mathbf{y}_t, \mathbf{x}_{t+k})$, to predict future sensor measurements along sampled trajectories, thus allowing the actor and critic to be evaluated within PAC-NMPC. An overview of our approach is shown in Figure 2.

We extend the PAC-NMPC algorithm in three major ways:

- 1) RL-based warm start of the decision variables.
- 2) Learned stochastic value function as terminal cost.
- 3) Value function improvement constraint along trajectory.

These contributions will be discussed in the subsequent subsections. Since the implementation and training of these learned models is specific to each robotic system, we leave discussion of these components to later sections.

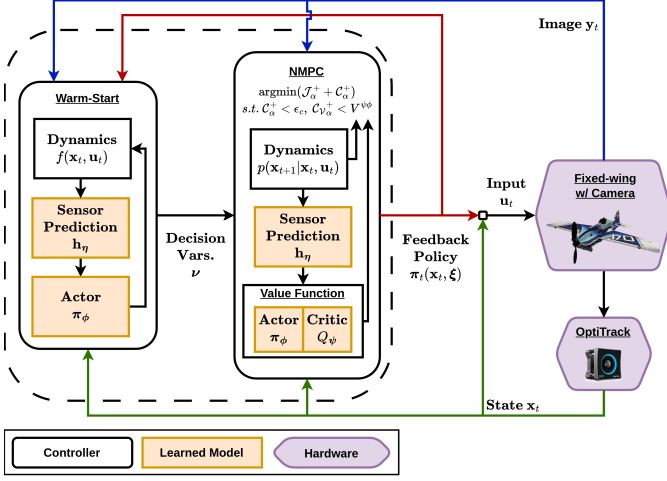


Fig. 2: Overview of AC-PAC-NMPC.

A. RL-based Warm Start

We initialize the PAC-NMPC decision variables using the learned actor policy. Since the RL actor policy is trained to minimize the expected cost-to-go, this warm-start may reduce optimization time to convergence and help avoid non-optimal local minima. For high-dimensional input spaces, we have found that this feature is important for keeping the SNMPC policy within the distribution of the trained RL model. Our warm-start approach is summarized in Algorithm 1.

Given an initial state, $\mathbf{x}_t$, and sensor measurement, $\mathbf{y}_t$, the initial input of the warm-start trajectory can be computed as $\hat{\mathbf{u}}_t = \pi^\phi(\mathbf{x}_t, \mathbf{y}_t)$. However, subsequent actions cannot be computed directly, since future states and sensor measurements are needed to evaluate the actor along the rest of the trajectory, $\hat{\mathbf{u}}_{t+k} = \pi^\phi(\mathbf{x}_{t+k}, \hat{\mathbf{y}}_{t+k})$. To address this, we compute future states recursively using a nominal, deterministic dynamics model, $\mathbf{x}_{t+1} = \mathbf{f}(\mathbf{x}_t, \mathbf{u}_t)$.

Unlike the robot state, future sensor measurements cannot be simulated in the same way because they depend on an environment which is unknown *a priori*. Instead, future sensor measurements are estimated using a sensor prediction model, $\hat{\mathbf{y}}_{t+k} = \mathbf{h}^\eta(\mathbf{x}_t, \mathbf{y}_t, \mathbf{x}_{t+k})$. Rather than predicting the sensor model recursively, $\hat{\mathbf{y}}_{t+1} = \mathbf{h}^\eta(\mathbf{x}_t, \mathbf{y}_t, \mathbf{x}_{t+1})$, we predict each future sensor measurements at time $t+k$ directly from the initial sensor measurement. Empirically, we found that recursively applying sensor prediction led to error accumulation and degraded sensor measurement prediction.

The resulting actor warm-start initializes the mean of the SNMPC policy's surrogate distribution, μ , while the variance, Σ , is set to a prespecified high variance to promote exploration. Due to runtime limitations, it is often not possible for PAC-NMPC to fully reduce this exploration variance. However, because we ultimately execute the maximum likelihood estimate of the SNMPC policy, we can perform a final PAC-NMPC policy optimization iteration in which the variances are reduced to a prespecified small value. This produces PAC guarantees that more accurately reflect the executed policies and are often tighter.

While PAC-NMPC can often find a feasible solution with this actor warm-start, when the warm-start produces a trajectory that is severely infeasible (e.g., due to distribution

Algorithm 1: Warm Start

Input: $\mathbf{x}_t, \mathbf{y}_{(t-N_y):t}$

- 1 $\hat{\mathbf{u}}_t = \pi^\phi(\mathbf{x}_t, \mathbf{y}_t)$;
- 2 **for** $i = t, \dots, t + N_T - 1$ **do**
- 3 $\mathbf{x}_{i+1} = \mathbf{f}(\mathbf{x}_i, \hat{\mathbf{u}}_i)$;
- 4 $\hat{\mathbf{y}}_{i+1} = \mathbf{h}^\eta(\mathbf{x}_i, \mathbf{y}_i, \mathbf{x}_{i+1})$;
- 5 $\hat{\mathbf{u}}_{i+1} = \pi^\phi(\mathbf{x}_{i+1}, \hat{\mathbf{y}}_{i+1})$;
- 6 **if** $c(\mathbf{x}_{i+1}, \hat{\mathbf{u}}_{i+1}, \mathbf{y}_{(t-N_y):t}) > 0$ **then**
- 7 warm-start from prior policy;
- 8 **return**
- 9 warm-start from $\mu = (\hat{\mathbf{u}}_t, \dots, \hat{\mathbf{u}}_{t+N_T-1})$;

shift), the PAC-NMPC policy distribution may not sample any feasible trajectories. The lack of feasible trajectories in this case will result in failed optimization. To improve out-of-distribution performance, we evaluate the degree to which the actor policy warm-start violates constraints. If it does, we instead default to warm-starting using the previously optimized PAC-NMPC policy, as done in prior research [60].

B. Uncertainty-Aware RL-Augmented Cost

A key limitation of NMPC is the potential myopic behavior from optimizing over a finite horizon. For online planning scenarios, this horizon is frequently restricted by computational resources, especially for robots characterized by complex dynamics and large state spaces. Taking inspiration from [36], we use the learned value function as the terminal cost

$$\ell_f(\mathbf{x}_{t+N_T}) = V^{\phi\psi}(\mathbf{x}_{t+N_T}, \hat{\mathbf{y}}_{t+N_T}) \quad (13)$$

where $V^{\phi\psi}$ is composed of the actor, π^ϕ , and critic, Q^ψ . A key difference between our approach and [36] is that our value function is dependent on sensor measurements and, therefore, we predict the terminal sensor measurement, $\hat{\mathbf{y}}_{t+N_T} = \mathbf{h}^\eta(\mathbf{x}_t, \mathbf{y}_t, \mathbf{x}_{t+N_T})$. Consequently, the resulting trajectory cost approximates the infinite-horizon perception-based cost-to-go given the current state and sensor measurements.

Also similar to [36], we learn an approximate distribution over value functions. However, while [36] did this to facilitate exploration, we do this so that PAC-NMPC can reason not only about the uncertainty in the robot dynamics, but also the uncertainty in the value function, during run-time execution.

To model the stochastic value function, we implement π^ϕ , Q^ψ , and $\mathbf{h}^\eta$ as stochastic models by representing these models as neural networks trained with Monte Carlo (MC) dropout:

$$r_j^{(l)} \sim \text{Bernoulli}(p) \quad \tilde{\mathbf{y}}^{(l)} = \mathbf{r}^{(l)} * \mathbf{y}^{(l)} \quad (14)$$

where $l \in \{1, \dots, L\}$ indexes the hidden layers, $\mathbf{r}^{(l)}$ is the vector of independent Bernoulli random variables that have a probability p of being 1, $\mathbf{y}^{(l)}$ denotes the vector of outputs from layer l , $*$ denotes an element-wise product, and $\tilde{\mathbf{y}}^{(l)}$ denotes the post-dropout outputs. Dropout was originally proposed as a way of reducing neural network overfitting [68], but has also been shown to approximate a Bayesian neural network when used during inference [69].

One traditional shortcoming of MC dropout is that the network must be inferenced many times with a different dropout mask for each forward pass, thus significantly increasing inference time. However, since PAC-NMPC must already

inference these networks for each sampled trajectory, applying a different dropout mask for each trajectory results in no additional forward passes. Thus, while other approaches can be used to model uncertainty, such as ensembles and multi-headed networks, we select MC dropout since it incurs both a negligible training time and inference time cost.

We note that, for simple (e.g., 2D) environments and sensors with large fields-of-view, a learned sensor prediction model may not be necessary for successful navigation.

C. Uncertainty-Aware Value Function Improvement Constraint

While using the learned value function as the terminal cost encourages less myopic behavior, it does not guarantee that the resulting policy will actually reduce the cost-to-go. Therefore, we also apply a value function improvement constraint along the trajectory.

We introduce an additional PAC-bound, $\mathcal{C}_{\nu_{\alpha}^+}(\nu)$, on the learned value function at the terminal state such that

$$\mathbb{P}(\mathbb{E}[V^{\phi\psi}(\mathbf{x}_{t+N_T}, \hat{\mathbf{y}}_{t+N_T})] \leq \mathcal{C}_{\nu_{\alpha}^+}(\nu)) \geq 1 - \delta. \quad (15)$$

We constrain the PAC-NMPC optimization problem such that this bound must be less than the learned value function at the initial state of the trajectory

$$\text{s.t. } \mathcal{C}_{\nu_{\alpha}^+}(\nu) \leq \mathbb{E}[V^{\phi\psi}(\mathbf{x}_t, \mathbf{y}_t)]. \quad (16)$$

Since in practice $\mathcal{C}_{\nu_{\alpha}^+}(\nu)$ has a nonzero upper-bound gap such that $\mathbb{E}[V^{\phi\psi}(\mathbf{x}_{t+N_T}, \hat{\mathbf{y}}_{t+N_T}) - V^{\phi\psi}(\mathbf{x}_t, \mathbf{y}_t)] < 0$ and because this learned value function approximates the cost-to-go, satisfaction of this constraint indicates that the robot is expected to reduce the cost-to-go over the finite planning horizon. Similarly to the prior section, we sample from the stochastic learned value function to incorporate model uncertainty into this PAC bound. In contrast to prior research [6], which bundled the value function improvement condition into the probability of constraint violation bound, this approach provides two distinct probabilistic guarantees.

The resulting optimization problem is

$$\begin{aligned} \nu^* &= \arg \min_{\nu} \min_{\alpha > 0} (\mathcal{J}_{\alpha}^+(\nu) + \gamma \mathcal{C}_{\alpha}^+(\nu)) \\ \text{s.t. } \mathcal{C}_{\alpha}^+(\nu) &\leq \epsilon_c. \\ \mathcal{C}_{\nu_{\alpha}^+}(\nu) &\leq \mathbb{E}[V^{\phi\psi}(\mathbf{x}_t, \mathbf{y}_t)], \end{aligned} \quad (17)$$

where ϵ_c is the maximum allowable probability of constraint violation. Also, in contrast to [6], both PAC bounds are imposed as hard constraints in the optimization problem. We leave the constraint violation bound as a penalty in the optimization function itself, as it is often beneficial to further minimize the violation bound below ϵ_c whenever possible. We solve this optimization problem using SNOPT [70], an SQP algorithm for constrained optimization.

VI. RALLY CAR UGV WITH LIDAR: APPROACH AND EVALUATION

In this section, we discuss the application of AC-PAC-NMPC to a 1/10th scale rally car platform equipped with a LiDAR sensor (Fig. 3). This section follows the formulation and includes experimental results presented in [6]. We

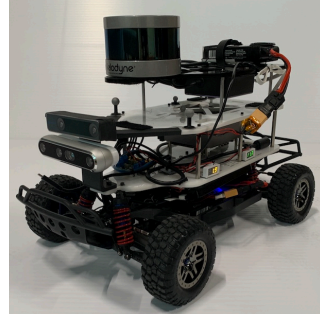


Fig. 3: 1/10th scale rally car platform with LiDAR sensor.

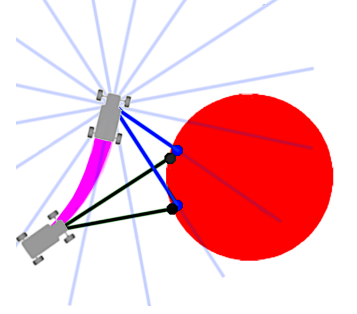


Fig. 4: Geometric LiDAR prediction.

include this summary of the formulation and experimental results to provide a self-contained presentation of the approach across multiple robotic platforms and to establish a basis for comparison against the more complex system presented in Section VII. In particular, because this prior research does not leverage the RL warm-start and learned perception prediction components described in the prior section, it helps demonstrate how these components become more critical for dynamical systems with larger states spaces, larger measurement spaces, and a diminished field-of-view.

A. Rally Car Dynamics and LiDAR Sensor

The rally car dynamics are represented by a stochastic bicycle model. The state is given as $\mathbf{x}_t = [r_x \ r_y \ \theta \ v \ \delta_s]^T$ where r_x, r_y denote the position, θ is the orientation, v the forward speed, and δ_s the steering angle. The input is given as $\mathbf{u}_t = [\dot{v} \ \dot{\delta}_s]^T$ where $\dot{v}$ is the forward acceleration and $\dot{\delta}_s$ is the steering angle rate of change.

The continuous-time, nominal dynamics are given as

$$\mathbf{f}(\mathbf{x}_t, \mathbf{u}_t) = [v \cos(\theta), v \sin(\theta), \frac{v \tan(\delta_s)}{l}, \dot{v}, \dot{\delta}_s]^T \quad (18)$$

where $l = 0.33\text{m}$ is the wheelbase. The stochastic, discrete-time dynamics applies Gaussian noise and Euler integration with a process covariance of $\Sigma_f = \text{diag}([4e^{-4}, 4e^{-4}, 1.1e^{-2}, 1e^{-1}, 5.6e^{-3}])$, which was fit from hardware data. There are limits on the acceleration, $\dot{v} \in [-1, 1] \frac{\text{m}}{\text{s}^2}$, the steering rate, $\dot{\delta}_s \in [-1, 1] \frac{\text{rad}}{\text{s}}$, and the steering angle, $\delta_s \in [-0.4, 0.4] \text{rad}$.

The rally car was equipped with a 64 beam, 360° planar LiDAR, which returned range measurements $\mathbf{y}_t = [y_t^0 \ y_t^1 \ \dots \ y_t^{63}]$ at bearings $\beta = [\beta^0 \ \beta^1 \ \dots \ \beta^{63}]$.

B. Actor-Critic

Using the dynamics and sensor model described above, the actor and critic networks were trained using the CleanRL [71] implementation of TD3 [67]. During training, the stage cost was summed with a constraint violation penalty:

$$g(\mathbf{x}_t, \mathbf{u}_t) = \ell(\mathbf{x}_t, \mathbf{u}_t) + \gamma_c c(\mathbf{x}_t, \mathbf{u}_t) \quad (19)$$

where $\gamma_c = 1000$ was a heuristically selected penalty. Unlike the stage constraint used during planning (Def. III.5), the training constraint, $c(\mathbf{x}_t, \mathbf{u}_t) \in \{0, 1\}$, utilizes full environmental knowledge, and $c(\mathbf{x}_t, \mathbf{u}_t) = 1$ indicates constraint violation.

The actor and critic, $\pi^\phi(\mathbf{z}_t)$ and $Q^\psi(\mathbf{z}_t, \mathbf{u}_t)$ respectively, were parameterized by fully connected neural networks with two hidden layers, 256 neurons per layer, 10% dropout, and ReLU activation functions. To aid efficient learning, the observations were preprocessed before being input into the networks. The network inputs consisted of the speed, tangent of the steering angle, range to the goal, cosine and sine of the bearing to the goal, and the LiDAR measurement, which were all normalized. Training environments, which included obstacles, initial states, and goal states, were randomly sampled. These networks were trained until the actor loss converged.

C. Sensor Prediction

Because these experiments were restricted to a two-dimensional environment with planar dynamics and a sensor model with a 360° field-of-view, sensor prediction for this system could be performed geometrically and did not incorporate prediction uncertainty quantification. This approach is visualized in Figure 4.

Given the current state, $\mathbf{x}_t$, each valid LiDAR return was projected into an observed obstacle point $\mathbf{o}^j$:

$$\mathbf{o}^j = \begin{bmatrix} r_{xt} \\ r_{yt} \end{bmatrix} + \begin{bmatrix} \cos(\theta_t + \beta^j) \\ \sin(\theta_t + \beta^j) \end{bmatrix} y_t^j. \quad (20)$$

Then, the range and bearing to each of these points from the future state $\mathbf{x}_{t+i}$ was calculated as

$$\hat{y}^j = \|\mathbf{o}^j - r_{xt+i} \ r_{yt+i}\|^T \quad (21)$$

$$\hat{\beta}^j = \text{atan2}(\mathbf{o}^j - [r_{xt+i} \ r_{yt+i}]^T) - \theta_{t+i}. \quad (22)$$

Since these bearing estimates do not necessarily correspond to the discrete bearing values used by the sensor, the estimated LiDAR measurements were assigned to the closest bearing:

$$\hat{y}_{t+i}^k = \hat{y}^j \quad k = \arg \min_k \{|\hat{\beta}^j - \beta^k|\}. \quad (23)$$

D. Simulation Experiments

AC-PAC-NMPC was compared against four baselines across two simulation environment distributions. Unique actor and critic networks were trained for each distribution. RL training and simulations were run on a laptop with an Intel Core i-9-13900H CPU and a Nvidia GeForce RTX 4080 Max-Q GPU.

The first baseline used PAC-NMPC with a quadratic terminal cost, $\ell_f(\mathbf{x}_{t+N_T}) = (\mathbf{x}_{t+N_T} - \mathbf{x}_G)^T \mathbf{Q}_f (\mathbf{x}_{t+N_T} - \mathbf{x}_G)$ where $\mathbf{Q}_f = \text{diag}([1 \ 1 \ 0 \ 0 \ 0])$. In the second baseline, LiDAR measurements were used to continuously build and maintain an occupancy grid of the environment using the widely adopted Nav2 framework [72]. Each planning iteration, the A* search algorithm was run on the grid to find the shortest path to the goal. A receding horizon goal, $\mathbf{x}_{A^*}$, was selected at a distance of $v_{max} \cdot N_T \cdot \Delta t$ along the path and was used to form a quadratic terminal cost, $\ell_f(\mathbf{x}_{t+N_T}) = (\mathbf{x}_{t+N_T} - \mathbf{x}_{A^*})^T \mathbf{Q}_f (\mathbf{x}_{t+N_T} - \mathbf{x}_{A^*})$. The third baseline evaluated the actor policy itself, $\pi^\phi(\mathbf{x})$. The fourth baseline optimized trajectories with MPPI [73], while still using the learned value function as a terminal trajectory cost.

The following experiments used a quadratic stage cost, $\ell(\mathbf{x}_t, \mathbf{u}_t) = (\mathbf{x}_t - \mathbf{x}_G)^T \mathbf{Q} (\mathbf{x}_t - \mathbf{x}_G)$, where $\mathbf{Q} =$

Approach		Success	Stuck	Violation
RL Actor Network		90%	3%	7%
MPPI w/ Learned $V^{\phi\psi}$		70%	7%	23%
PAC-NMPC	Quad. Term. Cost	76%	24%	0%
	Map & A*	89%	4%	7%
	Learned $V^{\phi\psi}$	97%	3%	0%

TABLE II: Simulated cluttered environments results.

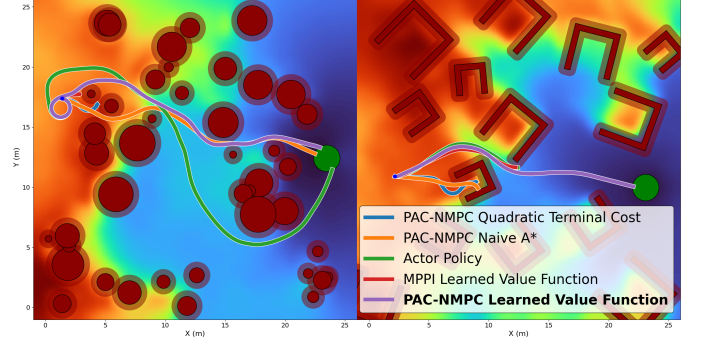


Fig. 5: Cluttered and trap environment examples. Slices of the learned value functions are visualized as heatmaps.

$\text{diag}([1e^{-2} \ 1e^{-2} \ 0 \ 0 \ 0])$. The stage constraint, $c(\mathbf{x}_t, \mathbf{u}_t, \mathbf{y}_t) \leq 0$, enforced bounds on the forward speed, $v \in [-1, 3] \frac{\text{m}}{\text{s}}$, and required the vehicle to maintain a minimum distance of 0.5m from all observed points from the latest LiDAR measurement. RL-based warm-starting was not used in these experiments.

PAC-NMPC optimized feedback policies over 12 timestep trajectories with $\Delta t = 0.1$ sec at a replanning period of $H = 0.2$ sec. The optimization used $L = 5$ prior policies with $M = 1024$ trajectory samples per prior with $\delta = 0.05$. The trajectory costs were normalized before optimization to achieve tighter PAC bounds. The constraint violation bound penalty was set to $\gamma = 2$ in the cluttered environments and $\gamma = 4$ in the concave trap environments. These were empirically determined to yield the best performance in these environments. When running MPPI, the sampled trajectory costs were similarly normalized and summed with a constraint violation indicator multiplied by γ . The same number of timesteps, Δt , H , and M were used, along with a temperature of $\gamma_t = 0.35$ and sampling variance of $\Sigma_\epsilon = 0.01$. Both PAC-NMPC and MPPI were allowed to run for as many iterations as possible within the replanning period and the resulting policies were interpolated to 50Hz.

1) *Cluttered Environments*: The first distribution of environments consisted of randomly placed circular obstacles with random radii. These obstacles were allowed to overlap, which allowed the constraint regions to combine to form complex environments. 100 testing environments were sampled from this distribution. Trivial environments in which no obstacles blocked the path to the goal were discarded.

Results are shown in Table II, with an example environment in Figure 5. When using a quadratic terminal cost, the system often got caught in local minima. When using A*, the system occasionally planned paths through obscured obstacles, which caused constraint violations if the LiDAR was unable to view the obscured obstacles until the system was too close to recover. The actor policy was unable to explicitly enforce constraints which resulted in occasional violations. Our approach outperformed all baselines and never violated the constraints.

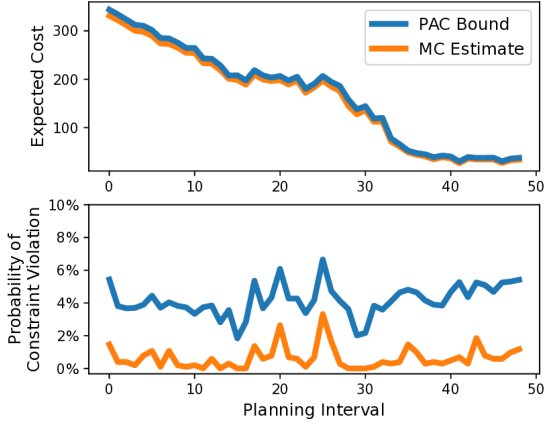


Fig. 6: PAC Bounds & Monte Carlo estimates.

Approach	Success	Stuck	Violation
RL Actor Network	82%	2%	16%
MPPI w/ Learned $V^{\phi\psi}$	55%	9%	36%
PAC-NMPC	Quad. Term. Cost	47%	53%
	Map & A*	91%	4%
	Learned $V^{\phi\psi}$	93%	7%

TABLE III: Simulated trap environments results.

Figure 6 displays a plot of the optimized PAC bounds at each planning interval for an entire example trial when using the learned value function as a terminal cost and constraint. The probability of collision remains non-zero throughout the trial due to dynamics uncertainty and increases as the system gets closer to obstacles. To validate PAC bounds, the bounds were compared against Monte Carlo estimates of the expected cost and probability of constraint violation, which were formed by sampling 1024 trajectories and dropout masks. On average, PAC-NMPC produced guarantees that the probability of constraint violation of the NMPC generated policies would be less than 5%.

To demonstrate that this approach incorporated uncertainty from the actor and critic networks into the PAC bound computation, the percentage of bound violations was evaluated when optimizing the bounds with and without sampling dropout masks. In both cases, these were compared against Monte Carlo estimates using 1024 sampled trajectories and dropout masks. Across all trials, when optimizing with sampled dropout masks, the expected cost bound was never violated, and the probability of constraint violation bound was violated in only 0.34% of planning intervals. When optimizing without sampled dropout masks, the expected cost bound was violated in 65.49% of planning intervals, and the probability of constraint violation bound was violated in 1.45%.

2) *Concave Trap Environments*: The second distribution of environments consisted of concave traps to highlight the ability of AC-PAC-NMPC to safely avoid local minima. The number of obstacles, side lengths of each obstacle, and pose of each obstacle was randomly sampled. The angles of the obstacles were sampled uniformly such that they were pointing towards the starting position of the robot $\pm \frac{\pi}{2}$ radians. 100 environments were sampled from the distribution and used to evaluate AC-PAC-NMPC, which outperformed all baselines and never violated the constraints (Table III). An example



Fig. 7: Hardware environment example.

Approach	Success	Stuck	Violation
Actor Policy	85%	0%	15%
PAC-NMPC w/ Learned $V^{\phi\psi}$	95%	5%	0%
Actor Policy w/ Mismatch	70%	0%	30%
PAC-NMPC w/ Learned $V^{\phi\psi}$ w/ Mismatch	80%	20%	0%

TABLE IV: Hardware environments results.

environment is shown in Figure 5.

E. Hardware experiments

AC-PAC-NMPC was evaluated on a 1/10th scale Traxxas Rally Car platform with a Velodyne Puck LITE LiDAR. An OptiTrack Motion Capture system was used for state estimation. 20 random environments consisting of a maximum of 8 circular obstacles in an 8 meter by 6 meter space were generated. Environments in which obstacles overlapped or in which no obstacles were blocking the path to the goal were discarded. An example environment is shown in Figure 7.

The value function used in these hardware experiments was trained entirely in the cluttered environment simulation. In contrast to the simulated LiDAR, the Velodyne Puck LITE LiDAR has noise and returns range measurements to the walls. Thus, these environments are outside of the training distribution. Additionally, a value function was also trained with an incorrect wheelbase, 0.5m, to demonstrate the utility of this approach even in the presence of model mismatch.

AC-PAC-NMPC outperformed the actor policy and never collided with obstacles (Table IV). This indicates that utilizing the RL models inside PAC-NMPC provided better robustness for the sim-to-real transfer. When using the learned value function trained with the incorrect wheelbase, but using the correct wheelbase when sampling trajectories, AC-PAC-NMPC outperformed the actor policy and never collided with obstacles. Thus, it was able to safely utilize an RL model trained in the presence of model mismatch.

VII. FIXED-WING UAV WITH DEPTH CAMERA: APPROACH AND EVALUATION

In this section, we apply AC-PAC-NMPC to a 32-inch wing-span fixed-wing aerial vehicle equipped with a depth camera (Fig. 8). This system has significantly more complex dynamics and sensing than the rally car in Section VI. Consequently, the novel algorithmic contributions presented in Section V, including the RL-based warm start, learned sensor prediction

model, and separate value function improvement PAC bound, are critical for enabling the successful navigation.

A. Fixed-wing Dynamics and Depth Camera Sensor

We use the dynamics model proposed in [57] with a nonunit-quaternion orientation representation [74].

The state and control are defined as

$$\mathbf{x}_t = [\mathbf{r}^T \mathbf{q}^T \mathbf{v}^T \boldsymbol{\omega}^T \boldsymbol{\delta}_s^T \delta_{th}]^T \quad \mathbf{u}_t = [\mathbf{u}_s^T u_{th}]^T \quad (24)$$

where $\mathbf{r} \in \mathbb{R}^3$ is the position of the center of mass in the world frame, $\mathbf{q} \in \mathbb{H}$ is the orientation, $\mathbf{v} \in \mathbb{R}^3$ is the linear velocity in the world frame, $\boldsymbol{\omega} \in \mathbb{R}^3$ is the angular velocity in the body frame, $\boldsymbol{\delta}_s = [\delta_a, \delta_e, \delta_r] \in \mathcal{D} \subset \mathbb{R}^3$ are the control surface angles, $\delta_{th} \in \mathbb{R}$ is the propeller thrust magnitude, $\mathbf{u}_s \in \mathbb{R}^3$ are the control surface angular rates, and $u_{th} \in \mathbb{R}$ is the throttle. The continuous-time, nominal dynamics are defined as

$$\dot{\mathbf{x}}_t = \mathbf{f}(\mathbf{x}_t, \mathbf{u}_t) = [\dot{\mathbf{r}}^T \dot{\mathbf{q}}^T \dot{\mathbf{v}}^T \dot{\boldsymbol{\omega}}^T \dot{\boldsymbol{\delta}}_s^T \dot{\delta}_{th}]^T$$

$$\dot{\mathbf{r}} = \mathbf{v} \quad \dot{\boldsymbol{\omega}} = \mathbf{J}^{-1}(\mathbf{m} - \boldsymbol{\omega} \times \mathbf{J}\boldsymbol{\omega})$$

$$\dot{\mathbf{q}} = \frac{1}{2}\boldsymbol{\Omega}(\boldsymbol{\omega})\mathbf{q} + 0.1(1 - \mathbf{q}^T\mathbf{q})\mathbf{q} \quad \dot{\boldsymbol{\delta}}_s = \mathbf{u}_s$$

$$\dot{\delta}_{th} = a\delta_{th} + bu_{th} \quad (25)$$

where $\mathbf{R}(\mathbf{q}) \in SO(3)$ is the rotation matrix, m is the mass, $\mathbf{J}$ is the inertia, a and b define the thrust dynamics, and

$$\boldsymbol{\Omega}(\boldsymbol{\omega}) = \begin{bmatrix} 0 & -\omega_0 & -\omega_1 & -\omega_2 \\ \omega_0 & 0 & \omega_2 & -\omega_1 \\ \omega_1 & -\omega_2 & 0 & \omega_0 \\ \omega_2 & -\omega_1 & -\omega_0 & 0 \end{bmatrix}.$$

The forces acting on the fixed-wing in the body-fixed frame can be written as

$$\mathbf{f} = \sum_i (\mathbf{R}_{s_i} \mathbf{f}_{s_i}) + \delta_{th} \mathbf{R}_t \mathbf{e}_x - mg \mathbf{R}(\mathbf{q})^T \mathbf{e}_z + \mathbf{f}_d \quad (26)$$

where $\mathbf{R}_{s_i}$, $\mathbf{R}_t \in SO(3)$ are the rotation of the aerodynamic surfaces and the thrust source, respectively, with respect to the body-fixed frame, $\mathbf{e}_x$, $\mathbf{e}_z$ are unit vectors in the x and z directions, respectively, g is gravity, $\mathbf{f}_{s_i}$ is the force from each aerodynamic surface, and $\mathbf{f}_d$ is the drag force.

The forces from the i^{th} aerodynamic surface are modeled using the flat plate model in [75]:

$$\mathbf{f}_{s_i}(\mathbf{x}_t, \mathbf{u}_t) = \rho S_i \|\mathbf{v}_{s_i}(\mathbf{x}_t, \mathbf{u}_t)\| (\mathbf{v}_{s_i}(\mathbf{x}_t, \mathbf{u}_t) \cdot \mathbf{e}_z) \mathbf{e}_z \quad (27)$$

where ρ is the air density, S_i is the surface area, and $\mathbf{v}_{s_i}$ is the surface velocity given as

$$\mathbf{v}_{s_i} = \mathbf{R}_{s_i}^T (\mathbf{v}_b + \boldsymbol{\omega} \times \mathbf{r}_{h_i} + \gamma_i \mathbf{v}_{bw}) + (\mathbf{R}_{s_i}^T \boldsymbol{\omega} + \boldsymbol{\omega}_{s_i}) \times \mathbf{r}_{s_i}. \quad (28)$$

Here, $\mathbf{v}_b = \mathbf{R}(\mathbf{q})^T \mathbf{v}$ is the linear velocity in the body-fixed frame, $\mathbf{r}_{h_i}$ is displacement from the center of mass to the stationary point on the surface with respect to the body frame origin, $\mathbf{r}_{s_i}$ is the displacement from the hinge point to the surface center of pressure, and $\boldsymbol{\omega}_{s_i}$ is the commanded surface rotation rate, and γ_i is an empirically determined backwash coefficient. $\mathbf{v}_{bw}$ is the backwash velocity from the propeller, which is approximated using actuator disk theory

$$\mathbf{v}_{bw} = \left[\sqrt{\|\mathbf{v}_p\|^2 + \frac{2\delta_{th}}{\rho S_{disk}}} - \|\mathbf{v}_p\| \right] \mathbf{e}_x \quad (29)$$



Fig. 8: Edge540 aerial vehicle with RealSense D450 depth camera.

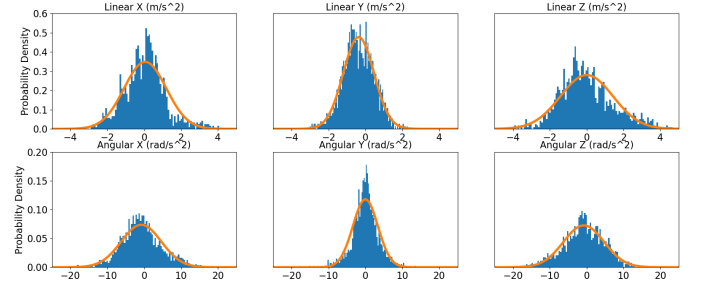


Fig. 9: Histogram of fixed-wing aerial vehicle model acceleration errors. Fitted Gaussian distribution shown in orange.

where $\mathbf{v}_p$ is the freestream velocity at the propeller and S_{disk} is the area of the actuator disk. The drag force is modeled as

$$\mathbf{f}_d = -\frac{1}{2} C_{b_d} \rho \|\mathbf{v}_b\| \mathbf{v}_b \quad (30)$$

where C_{b_d} is determined empirically.

The moment in the body-fixed frame, $\mathbf{m}$, is given as

$$\mathbf{m} = \sum_i ((\mathbf{r}_{h_i} + \mathbf{R}_{s_i} \mathbf{r}_{s_i}) \times \mathbf{R}_{s_i} \mathbf{f}_{s_i}) \quad (31)$$

The stochastic, discrete-time dynamics applies a second-order Runge-Kutta (RK2) integration and Gaussian process noise, fit from hardware flight data (Fig. 9).

The fixed-wing is equipped with a depth camera, which has an $87^\circ \times 58^\circ$ field of view. The sensor measurement, $\mathbf{y}_t$, is a flattened depth image.

B. Actor Critic Training

Similarly to Section VI-B, the actor and critic were trained using the CleanRL [71] implementation of TD3 [67]. The dynamics and sensor model previously described were used for training, and the stage cost was summed with a constraint violation penalty, $\gamma_c = 1500$. The actor and critic, $\pi^\phi(\mathbf{z}_t)$, $Q^\psi(\mathbf{z}_t, \mathbf{u}_t)$ were parameterized identically as fully connected neural networks with two hidden layers, 256 neurons per layer, 10% dropout, and ReLU activation functions.

The depth camera sensor measurement, $\mathbf{y}_t$, was limited to a resolution of 16×12 pixels. This low resolution enables rapid inference of the actor, critic, and sensor prediction models. Additionally, given the simplicity of these inputs compared to a high-resolution image, the learned models converge faster and generalize better. While this resolution may limit the ability of the system to detect thin obstacles, we found it

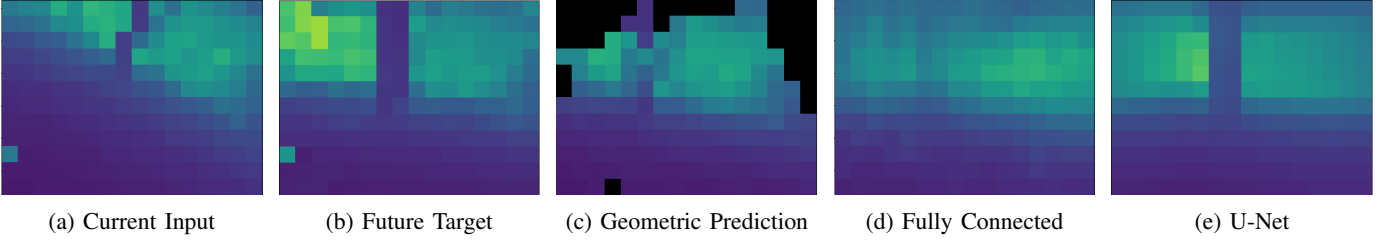


Fig. 10: Image prediction on decimated RealSense D450 depth image collected during fixed-wing flight. Depth is visualized as blue (near) to yellow (far).

sufficient for navigation in our targeted environments, where obstacles are 0.2 to 0.4 meters in radius.

To aid in efficient learning, the observations were preprocessed for the networks. The network inputs consisted of the z position, flattened rotation matrix, control surface angles, body-frame linear and angular velocities, range to goal, cosine and sine of the azimuth and elevation angles to the goal, and flattened depth image. These inputs were all normalized.

C. Sensor Prediction

As discussed in Section V, estimates of future sensor measurements, $\hat{\mathbf{y}}_t$, must be obtained to warm-start PAC-NMPC with the actor policy and to evaluate the value function at the terminal states of sampled trajectories. We first evaluated image prediction through geometric projection of current measurements to future states, as was done for LiDAR measurement prediction for the rally car platform.

First, the depth image was projected into 3D space, then assigned to pixels in the camera view at the future state. We evaluated both nearest pixel assignment and bilinear splatting. We found that this approach did not yield consistently accurate predictions and often left many pixels unassigned, making it unsuitable for evaluating the actor and critic at future states along sampled trajectories. Thus, we propose training a model to predict future state measurements with supervised learning.

We generated a training dataset entirely from simulation using the RL actor policy. The actor policy controlled the system through randomly sampled environments, yielding a sequence of states, $(\mathbf{x}_0, \mathbf{x}_1, \dots)$, and sensor measurements $(\mathbf{y}_0, \mathbf{y}_1, \dots)$. For each timestep t in each of these environments, we added to the dataset the model inputs $(\mathbf{x}_t, \mathbf{y}_t, \mathbf{x}_{t+k})$ and targets $(\mathbf{y}_{t+k})$ for all $k \in [0, \dots, N_T]$. This produced a dataset that was well suited to predicting future sensor measurements along the entire trajectory horizon.

We then trained and evaluated fully connected and U-Net convolutional [76] networks and found that the U-Net architecture yielded superior results (Table V, Fig. 10). These networks were trained on a smooth L1 loss plus an SSIM Loss [77] and were evaluated on a 10% held-out validation set.

The fully connected network had two hidden layers, 4096 neurons per layer, SiLU activation functions, and 20% dropout. This had a similar computational complexity, measured in FLOPs, as the U-Net architecture that we evaluated.

The U-Net architecture had 3 encoder-decoder stages. The input depth image, $\mathbf{y}_t$, was normalized and passed through an initial convolutional layer which increases the number of

Model	FLOPs (M)	δ_1	δ_2	δ_3	REL	RMSE
Geometric (Nearest)	N/A	0.133	0.217	0.306	2.838	0.634
Geometric (Splat)	N/A	0.186	0.309	0.419	2.240	0.548
Fully Connected	36.798	0.848	0.935	0.963	0.170	0.081
U-Net	42.283	0.923	0.965	0.983	0.087	0.058

TABLE V: Performance of depth image prediction models on simulated data. δ_i is the percentage of pixels satisfying $\max(\frac{\hat{d}}{d}, \frac{d}{\hat{d}}) < 1.25^i$ where $\hat{d}$ and d are predicted and ground-truth depths. REL and RMSE are the mean absolute relative error and root mean squared error respectively.

image channels. We found 16 initial channels to produce adequate performance while allowing us to inference the network fast enough. Each stage consists of a convolutional layer, an RMS norm, a SiLU activation function, and 20% dropout. The number of image channels is doubled and the resolution is halved with each stage.

The initial and future states $\mathbf{x}_t, \mathbf{x}_{t+k}$ enter through the bottleneck of the network. First, the $SE(3)$ transform from the initial to future state, represented as a translation vector and rotation matrix, is computed and flattened. This transform is passed through a fully connected network with two hidden layers, 256 neurons, and SiLU activation functions. The resulting features are used to scale and shift the intermediate feature maps at the bottleneck of the network [78], thus conditioning the image prediction on the state transform. As is done with the actor and critic networks, we implement $\mathbf{h}^\eta$ as a stochastic model using MC dropout.

D. Simulation Experiments

The following experiments evaluated AC-PAC-NMPC on the fixed-wing aerial vehicle with a depth camera in challenging simulation environments. We compared our proposed approach against several baselines, performed an ablation study of the major novel algorithmic contributions, and evaluated our approach outside of the RL training distribution.

1) *Controller Configuration:* We used a quadratic stage cost

$$\begin{aligned} \ell(\mathbf{x}_t, \mathbf{u}_t) &= (\mathbf{x}_t - \mathbf{x}_G)^T \mathbf{Q} (\mathbf{x}_t - \mathbf{x}_G) + \mathbf{u}_t^T \mathbf{R} \mathbf{u}_t \\ \mathbf{Q} &= 0.1 \cdot \text{diag}([1 \ 1 \ 1 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 1 \ 0 \ 0 \ 0 \ 1 \ 1 \ 1 \ 0]) \\ \mathbf{R} &= 0.01 \cdot \text{diag}([1 \ 1 \ 1 \ 1]). \end{aligned} \quad (32)$$

The stage constraint, $c(\mathbf{x}_t, \mathbf{u}_t, \mathbf{y}_{(t-N_y):t}) \leq 0$, enforced bounds on the speed, $\|\mathbf{v}\| < 7.0 \frac{\text{m}}{\text{s}}$, the z position, $r_z \in [1.25, 3.25]$ and required the vehicle to maintain a minimum distance of 0.6m from all observed obstacle points from the latest depth images. Since the depth camera has a limited field

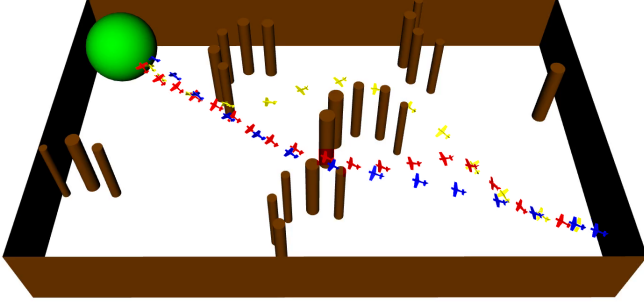


Fig. 11: Simulation environment example. Ceiling/floor not shown. RL actor in yellow, PAC-NMPC with global planner in red, AC-PAC-NMPC in blue.

of view, recently observed obstacles may leave the image as the aerial vehicle performs aerobatic maneuvers. To account for this, the obstacle constraint utilizes a history of depth images. If the queried state lies outside of the latest depth image field of view, an image from 0.1 sec prior is queried. This continues for a history of up to $N_y = 10$ prior depth images.

We optimized feedback policies over 11 timestep trajectories with $\Delta t = 0.1$ sec at a replanning period of $H = 0.1$ sec. In contrast to the rally car's configuration, we replan at every integration timestep. We found that this increased replanning rate allowed for improved reactivity to obscured obstacles given the increased vehicle speed and limited camera field of view. The optimization used $L = 1$ prior policies, $M = 1024$ trajectory samples per prior, and $\delta = 0.05$. The trajectory costs were normalized before optimization to achieve tighter PAC bounds. The constraint violation bound penalty was set to $\gamma = 2$ and the allowable threshold to $\epsilon_c = 0.1$.

2) *Environments*: We generated a distribution of environments consisting of randomly placed cylindrical obstacles in a rectangular room. The room width, length, and height were uniformly sampled between $[10, 30]$, $[20, 30]$, and $[4, 5]$ meters, respectively. These obstacles were arranged in randomly placed semi-circle arrangements that create concave traps. These traps create challenging environments for the fixed-wing aerial vehicle, which is unable to reliably stop and turn around when caught in the traps. Consequently, successful navigation often requires long-range reasoning beyond the finite MPC horizon. The number of semi-circle traps was randomly sampled between 0 and 6, and were oriented towards the initial position of the plane, $\pm \frac{\pi}{2}$. The obstacle radii were sampled between 0.2 and 0.4 meters. Obstacles were not allowed to overlap. The simulated depth camera included observations of the cylindrical obstacles, walls, ceiling, and floor. An example environment is shown in Figure 11.

3) *Baseline Comparison*: We compared our approach against several baselines in these simulated environments. First, we compared against the RL actor policy itself, which was trained on this distribution of environments. Then, we compare against a sequence of increasingly complex variations of PAC-NMPC, each introducing an additional capability. These baselines isolate and compare the contributions of mapping, global planning, and unknown space penalties against

Approach		Success	Cost
RL Actor Network		81%	468.1
PAC-NMPC	Depth Image History	52%	757.7
	Map	63%	810.2
	Map & A*	74%	738.8
	Map & A* Unknown	85%	718.14
	Actor-Critic (ours)	90%	513.3

TABLE VI: Baseline Comparison Results

uncertainty-aware RL augmentation.

First, we evaluated PAC-NMPC using a history of depth images for its constraint and a quadratic terminal cost:

$$\begin{aligned}
 \ell_f(\mathbf{x}_t, \mathbf{u}_t) &= (\mathbf{x}_t - \mathbf{x}_G)^T \mathbf{Q} (\mathbf{x}_t - \mathbf{x}_G) \\
 &+ (\boldsymbol{\eta}_t - \boldsymbol{\eta}_G)^T \mathbf{Q}_\eta (\boldsymbol{\eta}_t - \boldsymbol{\eta}_G) \\
 &+ (\mathbf{v}_t - \mathbf{v}_G)^T \mathbf{Q}_v (\mathbf{v}_t - \mathbf{v}_G), \\
 \mathbf{d}_t &= \mathbf{r}_G - \mathbf{r}_t, \quad \boldsymbol{\eta}_G = \text{eul}(\mathbf{d}_t), \quad \mathbf{v}_G = v_G \cdot \mathbf{d}_t / \|\mathbf{d}_t\|, \\
 \mathbf{Q} &= \text{diag}([1 \ 1 \ 1 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 1 \ 0 \ 0 \ 0 \ 0.1 \ 0.1 \ 0.1 \ 0]), \\
 \mathbf{Q}_\eta &= 10 \cdot \text{diag}([1 \ 1 \ 1]), \quad \mathbf{Q}_v = 0.1 \cdot \text{diag}([1 \ 1 \ 1]). \quad (33)
 \end{aligned}$$

Here, $\boldsymbol{\eta}_t$, $\mathbf{v}_t$ are the Euler angles and linear world velocity of the aerial vehicle at state $\mathbf{x}_t$. The desired Euler angles, $\boldsymbol{\eta}_G$, and velocity, $\mathbf{v}_G$, are constructed from the vector between the robot and the goal, $\mathbf{d}_t$, where the target speed is set to $v_G = 6 \frac{\text{m}}{\text{s}}$.

To improve awareness of previously observed obstacles, the next baseline maintains a map of the environment. We maintained an occupancy voxel grid, which was constructed and maintained using the the widely adopted Nav2 framework [72]. The voxel grid had a resolution of $0.1\text{m} \times 0.1\text{m} \times 0.1\text{m}$ and was used in place of the depth image history for the obstacle constraint function.

To improve the robot's ability to avoid local traps in the environment, we incorporated a global planner into the next baseline. We implemented a 3-dimensional A* global planner. Each planning iteration, the A* search algorithm was run on the voxel grid to find the shortest path to the goal. These paths were pruned, smoothed, and parameterized by velocity, as was done in [57]. Target velocity along the path is decreased as the path curvature increases: straight segments were assigned $6 \frac{\text{m}}{\text{s}}$ and segments with the maximum curvature were assigned $3 \frac{\text{m}}{\text{s}}$. This encourages the aerial vehicle to slow down, resulting in a tighter turning radius, as the curvature of the path increases. A receding horizon goal, $\mathbf{x}_{A^*}$ was selected at a time horizon of 1 sec along the path to form a quadratic terminal cost,

$$\begin{aligned}
 \ell_f(\mathbf{x}_t, \mathbf{u}_t) &= (\mathbf{x}_t - \mathbf{x}_{A^*})^T \mathbf{Q} (\mathbf{x}_t - \mathbf{x}_{A^*}) \\
 &+ (\boldsymbol{\eta}_t - \boldsymbol{\eta}_{A^*})^T \mathbf{Q}_\eta (\boldsymbol{\eta}_t - \boldsymbol{\eta}_{A^*}) \\
 &+ (\mathbf{v}_t - \mathbf{v}_{A^*})^T \mathbf{Q}_v (\mathbf{v}_t - \mathbf{v}_{A^*}), \\
 \mathbf{Q} &= \text{diag}([10 \ 10 \ 10 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 1 \ 0 \ 0 \ 0 \ 0.1 \ 0.1 \ 0.1 \ 0]), \\
 \mathbf{Q}_\eta &= 10 \cdot \text{diag}([1 \ 1 \ 1]), \quad \mathbf{Q}_v = \text{diag}([1 \ 1 \ 1]), \quad (34)
 \end{aligned}$$

where $\boldsymbol{\eta}_{A^*}$ and $\mathbf{v}_{A^*}$ are the Euler angles and linear world velocity of the receding horizon goal as parameterized by the smoothed A* global path.

To improve the robot's ability to avoid obscured obstacles in the environment, the last baseline applies an unknown-space penalty to the A* planner. Each cell in the voxel grid is assigned as either occupied, unoccupied, or unobserved based on ray casting from the depth camera poses. This baseline

RL-based Warm-start	Learned Prediction	Success	Cost
	✓	3%	1001.2
✓		11%	889.7
✓	✓	90%	513.3

TABLE VII: Ablation Study Results

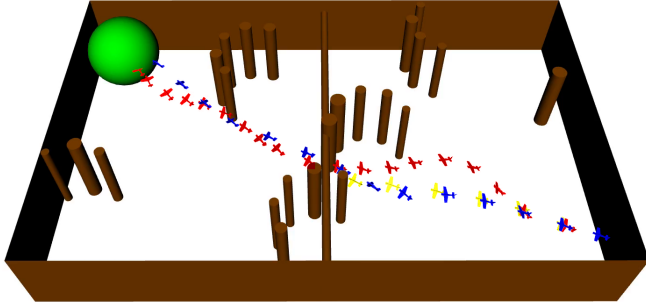


Fig. 12: Out-of-distribution simulation environment example. Ceiling/floor not shown. RL actor (collision) in yellow, PAC-NMPC with global planner in red, AC-PAC-NMPC in blue.

assigned an extra cost of 100 to the A* planner for traversing through unobserved cells.

Finally, our approach utilizes AC-PAC-NMPC without mapping or a global planner. We ran our experiments over 100 environments sampled from the training distribution. Trivial environments in which no obstacles blocked the shortest path to the goal were discarded.

We found that our approach had the highest success rate, 90%, of reaching the goal without violating the obstacle constraints (Table VI). Additionally, we found that our approach accomplished this with the lowest average accumulated cost over successful trials out of all the PAC-NMPC baselines. This indicates that in these environments, the RL guidance was most effective in enabling successful, low cost navigation, even outperforming PAC-NMPC with mapping and a global planner. Additionally, since our approach had a higher success rate than the RL actor alone, it was successful in improving the safety of the RL policy while benefiting from its improved long range behavior.

4) *Ablation Study*: We next performed an ablation study to evaluate the impact of two major contributions of this work: RL-based warm-start and learned sensor prediction. This study was run in the same sampled environments of the prior section. To evaluate RL-based warm-start, we replaced it with the default PAC-NMPC warm-start approach as presented in [60]. To evaluate the learned sensor prediction, we used geometric projection with nearest pixel assignment. We found that the lack of either of these components resulted in successful navigation in only a small percentage of trials (Table VII).

We believe that poor performance when lacking learned sensor prediction is caused by the inaccurate and unsuitable predictions generated by geometric projection, as demonstrated in Section VII-B. The RL-based warm-start was also particularly important for this system. One possible explanation is that without it, the critic network may be producing poor approximations of the value function. Because the critic is primarily trained on observation-input pairs generated by the

Approach		Success	Cost
RL Actor Network		46%	481.8
PAC-NMPC	Map & A* Unknown	76%	732.6
	Actor-Critic (ours)	76%	512.1

TABLE VIII: Out Of Distribution Experiment Results

actor policy, its estimates may be unreliable along trajectories that substantially deviate from the actor. This was not observed on the rally car with LiDAR, perhaps because its dynamics and sensor models have significantly lower dimensionality, thus reducing the impact of this effect. These results demonstrate the importance of these novel contributions when utilizing AC-PAC-NMPC to control more complex systems.

5) *Out of Distribution Experiment*: We evaluated the performance of our approach in environments that are outside of the RL training distribution. Since the actor and critic models were only trained in the presence of vertical cylindrical obstacles, we altered the testing environments by adding a horizontal obstacle. This is a relatively simple change to the testing environments for MPC, but may represent a significant distribution shift for the RL actor and critic models.

We reused the same testing environments from the previous subsections, but added a horizontal obstacle in the middle of each room. The horizontal obstacle was randomly sampled between heights of 1.25m and 3.25m. Obstacles sampled within 0.6m of the height of the goal were resampled since this blocks a large portion of the feasible flight space. The horizontal obstacle had a radius of 0.2m. An example environment is shown in Figure 12.

In the presence of these out-of-distribution obstacles, the RL actor policy violated obstacle constraints in most trials, dropping to a 46% success rate. Our approach, on the other hand, succeeded in 76% of trials, demonstrating improved ability to transfer to environments outside of the training distribution. In these environments, our approach matched the success rate of the best baseline, but achieved an improved average accumulated cost over successful trials (Table VIII). These results suggest that our approach is able to incorporate some of the improved long range behavior of the RL policy while being more robust to shifts from the training distribution.

E. Hardware Experiments

We evaluated our approach on a Twisted Hobbys 32-inch wing-span fixed-wing aerial vehicle equipped with an Intel RealSense D450 depth camera module and Intel Vision Processor D4 Board. An on-board Arduino Uno Q running a ROS2 docker image streamed the depth images to the controller laptop over WiFi at 50Hz using the realsense-ros wrapper.

The experiments took place in an Optitrack motion capture facility, which provided pose and velocity state estimates to the controller laptop at 180Hz over ethernet. The laptop had an Intel i9-13900H CPU and a Nvidia GeForce RTX 4080 Laptop GPU. The laptop executed the control algorithms and transmitted resulting control inputs to the aerial vehicle using a Futaba T6K transmitter.

Environment generation followed the same procedure as Section VII-D2, with a fixed room size and 1 to 2 sampled



Fig. 13: Example hardware environments with time-lapse trajectories of the aerial vehicle controlled by our approach.

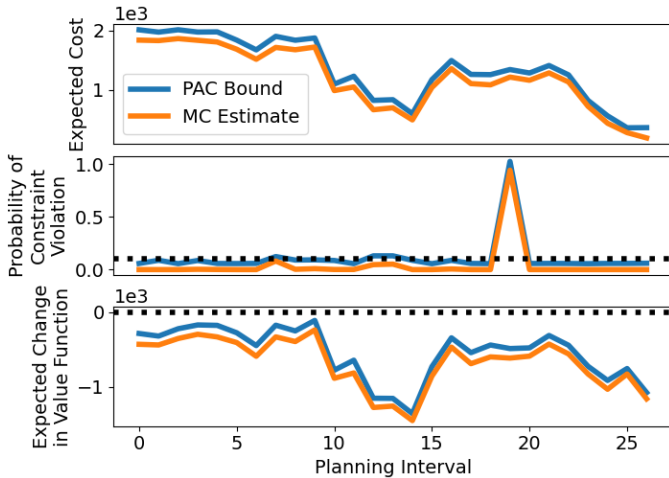


Fig. 14: Optimized PAC bounds compared to Monte Carlo estimates at each planning interval for one trial.

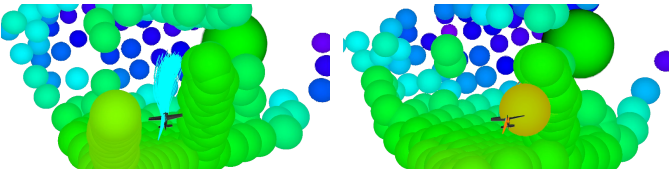


Fig. 15: Spurious noise in consecutive depth camera measurements, which caused a spike in C_α^+ .

semi-circle traps. I-beams and a spiral staircase were permanent obstacles across all environments. Play tunnels supported by PVC pipes were placed at the sampled obstacle positions. Example hardware environments are shown in Figure 13. We evaluated our approach in 10 randomly sampled environments.

We compared our approach against the RL actor and PAC-NMPC with mapping, an A* global planner, and unknown space costs. The actor, critic, and sensor prediction models were trained only in simulation and transferred to hardware without any additional fine-tuning. The depth images were decimated to 80×60 pixels on the Intel Vision Processor D4 Board. These images contained a lot of noise between foreground obstacles and the background. To reduce this noise, we ran a gradient filter on the Arduino Uno Q that invalidated pixels at which the local depth gradients exceeded 10% of the measured depth. When running PAC-NMPC with mapping, the images were further decimated to 40×30 pixels with a median filter to remove outliers. When running the RL actor and AC-PAC-NMPC, images were decimated to 16×12 pixels



Approach		Success	Cost
RL Actor Network		40%	403.8
PAC-NMPC	Map & A* Unknown	40%	325.4
	Actor-Critic (ours)	80%	252.2

TABLE IX: Fixed-wing aerial vehicle hardware results.

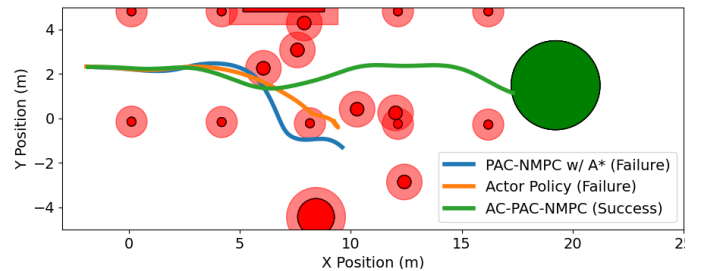


Fig. 16: Flight paths from one hardware trial.

(input resolution to the learned models) with the median filter.

We found that our approach had the highest success rate, 80%, and achieved the lowest average accumulated cost over successful trials (Table IX). Figure 14 displays the optimized PAC bounds at each planning interval for one of the environments. In this trial, there was a spike in the probability of constraint violation bound, C_α^+ , which was caused by spurious noise in the depth camera data (Fig. 15).

Both baselines only achieved a 40% success rate. The RL actor demonstrated a significantly lower success rate on hardware than in simulation. Two possible contributions are the mismatch of the modeled dynamics compared to the physical system and the addition of sensor noise. This suggests that our approach is better able to bridge the sim-to-real gap than RL policies alone.

The PAC-NMPC baseline also demonstrated significantly degraded hardware performance. During testing, we observed that noise in the depth camera still occasionally progressed to the mapping stage, despite the gradient filter and median decimation. Although the Nav 2 mapping algorithm attempts to remove false observations through ray casting, we observed that some intermittent noise remained in the map, which might have adversely affected planning performance. This may indicate that our approach, which limits the integration of sensor noise by only querying a history of depth images, could be more robust to this form of sensor noise.

Flight paths from one of the trials are shown in Figure 16. These results demonstrate that our approach successfully controlled, in real-time, a high-dimensional, nonlinear, robotic

system through an unknown environment using only local perception while providing probabilistic guarantees of safety.

VIII. ANALYSIS

In this section, through analysis of the value function and optimal policy in the presence of constraints, we provide theoretical justification for using actor and critic models within NMPC frameworks to explicitly enforce constraints.

Often, when training actor and critic models in the presence of constraints, the RL training cost, $g(\mathbf{x}_t, \mathbf{u}_t)$, is defined as

$$g(\mathbf{x}_t, \mathbf{u}_t) = \ell(\mathbf{x}_t, \mathbf{u}_t) + \gamma_c c(\mathbf{x}_t, \mathbf{u}_t) \quad (35)$$

where $\gamma_c > 0$ is a constraint violation penalty. For the purpose of this analysis, we will assume the training constraint is strictly non-negative, $c(\mathbf{x}_t, \mathbf{u}_t) \geq 0$, where $c(\mathbf{x}_t, \mathbf{u}_t) = 0$ indicates constraint satisfaction. The constraint violation penalty is either heuristically selected and static, or it can be interpreted as a Lagrange multiplier and learned during training, as is common in safe-RL approaches.

However, there is no guarantee that the resulting optimal policy will be constraint-free. Furthermore, there is no guarantee that a learned approximate policy will be constraint-free, especially outside of the training distribution. Through the following analysis, we show that, given a sufficiently large constraint penalty and the existence of a feasible policy, then for every policy generated by the optimal policy, there exists nearby control inputs that strictly satisfy the constraints while still descending the value function.

A. One-Step Feasible Descent

First, we prove that for a large enough γ_c , there exists some input in the neighborhood of the optimal policy that satisfies the constraints while descending the value function. We first show there exist non-optimal inputs in a local region around the optimal policy that still descend the value function.

Lemma VIII.1. *For a given $\mathbf{x}_t$, let the value function $V : \mathcal{X} \mapsto \mathbb{R}$ be continuous and bounded, $p(\mathbf{x}_{t+1} | \mathbf{x}_t, \mathbf{u}_t)$ be weakly continuous in $\mathbf{u}_t$, and $g(\mathbf{x}_t, \mathbf{u}_t) \succ 0$. Then, there exists a local region $B_\delta(\mathbf{u}_t^*) = \{\mathbf{u}_t \in \mathbb{R}^n : \|\mathbf{u}_t - \mathbf{u}_t^*\| < \delta\}$, such that $\mathbb{E}[V(\mathbf{x}_{t+1}) | \mathbf{x}_t, \mathbf{u}_t] - V(\mathbf{x}_t) < 0$.*

Proof. Given the optimal input $\mathbf{u}_t^*$, we can write

$$V(\mathbf{x}_t) = g(\mathbf{x}_t, \mathbf{u}_t^*) + \beta \mathbb{E}[V(\mathbf{x}_{t+1}) | \mathbf{x}_t, \mathbf{u}_t^*] \quad (36)$$

where $\beta \in (0, 1)$ and for some non-optimal input $\mathbf{u}_t$, we have

$$V(\mathbf{x}_t) \leq g(\mathbf{x}_t, \mathbf{u}_t) + \beta \mathbb{E}[V(\mathbf{x}_{t+1}) | \mathbf{x}_t, \mathbf{u}_t]. \quad (37)$$

We define expected one-step change in the value function as

$$\begin{aligned} \Delta V(\mathbf{x}_t, \mathbf{u}_t) &= \mathbb{E}[V(\mathbf{x}_{t+1}) | \mathbf{x}_t, \mathbf{u}_t] - V(\mathbf{x}_t) \\ &\geq (1 - \beta) \mathbb{E}[V(\mathbf{x}_{t+1}) | \mathbf{x}_t, \mathbf{u}_t] - g(\mathbf{x}_t, \mathbf{u}_t) \\ &\geq -g_\beta(\mathbf{x}_t, \mathbf{u}_t). \end{aligned} \quad (38)$$

We note that $\Delta V(\mathbf{x}_t, \mathbf{u}_t^*) = -g_\beta(\mathbf{x}_t, \mathbf{u}_t^*)$ and $g_\beta(\mathbf{x}_t, \mathbf{u}_t) \succ 0$ for sufficiently large β and $g(\mathbf{x}_t, \mathbf{u}_t)$. Given $V(\mathbf{x}_t)$ is continuous and bounded and $p(\mathbf{x}_{t+1} | \mathbf{x}_t, \mathbf{u}_t)$ is weakly continuous in $\mathbf{u}_t$, $\mathbb{E}[V(\mathbf{x}_{t+1}) | \mathbf{x}_t, \mathbf{u}_t]$ is continuous, and thus, $\Delta V(\mathbf{x}_t, \mathbf{u}_t)$ is continuous. Therefore, $\exists \delta > 0$ such that $\|\mathbf{u}_t - \mathbf{u}_t^*\| < \delta$ where $\Delta V(\mathbf{x}_t, \mathbf{u}_t) < 0$. $\square$

Next, under some assumptions, we show that as γ_c increases, expected constraint-to-go approaches lower bound ϵ .

Lemma VIII.2. *Let $g(\mathbf{x}_t, \mathbf{u}_t) = \ell(\mathbf{x}_t, \mathbf{u}_t) + \gamma_c c(\mathbf{x}_t, \mathbf{u}_t)$, where $\gamma_c > 0$, $\ell(\mathbf{x}_t, \mathbf{u}_t) \succ 0$, and $c(\mathbf{x}_t, \mathbf{u}_t) \geq 0$. For a given $\mathbf{x}_t$, suppose that $\Pi_\epsilon^c(\mathbf{x}_t) = \{\pi | \pi(\mathbf{x}_t) \in \mathcal{U}^c(\mathbf{x}_t), V_\pi^c(\mathbf{x}_t) < \epsilon\} \neq \emptyset$ where $\mathcal{U}^c(\mathbf{x}_t) = \{\mathbf{u}_t | c(\mathbf{x}_t, \mathbf{u}_t) = 0\}$, and $V_\pi^c(\mathbf{x}_t) = \mathbb{E}[\sum_{i=t}^\infty \beta^{i-t} c(\mathbf{x}_i, \pi(\mathbf{x}_i))]$. Then there exists a large enough γ_c such that $V_{\pi_\gamma^*}^c(\mathbf{x}_t) < \epsilon$, where π_γ^* is the optimal policy for a corresponding γ_c .*

Proof. We can write

$$V_{\pi_\gamma^*}^\ell(\mathbf{x}_t) = V_{\pi_\gamma^*}^\ell(\mathbf{x}_t) + \gamma_c V_{\pi_\gamma^*}^c(\mathbf{x}_t) \quad (39)$$

$$V_{\pi_\gamma^*}^\ell(\mathbf{x}_t) = \mathbb{E}\left[\sum_{i=t}^\infty \beta^{i-t} \ell(\mathbf{x}_i, \pi_\gamma^*(\mathbf{x}_i))\right] \quad (40)$$

$$V_{\pi_\gamma^*}^c(\mathbf{x}_t) = \mathbb{E}\left[\sum_{i=t}^\infty \beta^{i-t} c(\mathbf{x}_i, \pi_\gamma^*(\mathbf{x}_i))\right] \quad (41)$$

Since π_γ^* is optimal, $\forall \pi \in \Pi_\epsilon^c(\mathbf{x}_t)$

$$V_{\pi_\gamma^*}^\ell(\mathbf{x}_t) + \gamma_c V_{\pi_\gamma^*}^c(\mathbf{x}_t) \leq V_\pi^\ell(\mathbf{x}_t) + \gamma_c V_\pi^c(\mathbf{x}_t). \quad (42)$$

Since $V_{\pi_\gamma^*}^\ell(\mathbf{x}_t) \geq 0$

$$V_{\pi_\gamma^*}^c(\mathbf{x}_t) \leq \frac{1}{\gamma_c} V_\pi^\ell(\mathbf{x}_t) + V_\pi^c(\mathbf{x}_t) \quad (43)$$

$$V_{\pi_\gamma^*}^c(\mathbf{x}_t) < \frac{1}{\gamma_c} V_\pi^\ell(\mathbf{x}_t) + \epsilon \quad (44)$$

Therefore, for a large enough $\gamma_c < \infty$, $V_{\pi_\gamma^*}^c(\mathbf{x}_t) < \epsilon$. $\square$

Given the previous lemmas and a bound on ϵ , we show that there exists some feasible input in the neighborhood of the optimal policy that descends the value function.

Theorem VIII.1. *Let $\delta_{\min} = \min_{\pi \in \Pi_\epsilon^c} [\delta_\pi] > 0$ where δ_π is the neighborhood from Lemma VIII.1 for V_π . For the compact subset $\mathcal{U}_{\delta_{\min}} = \{\mathbf{u}_t \in \mathcal{U} \mid \text{dist}(\mathbf{u}_t, \mathcal{U}^c) \geq \delta_{\min}\}$, by the Extreme Value Theorem, $c_{\delta_{\min}} = \min_{\mathbf{u}_t \in \mathcal{U}_{\delta_{\min}}} [c(\mathbf{x}_t, \mathbf{u}_t)] > 0$.*

For a given $\mathbf{x}_t$, and the assumptions of Lemmas VIII.1 and VIII.2, if $\Pi_\epsilon^c \neq \emptyset$ with $\epsilon < c_{\delta_{\min}}$, then there exists a feasible input, $\mathbf{u}_t^c \in \mathcal{U}^c(\mathbf{x}_t)$, in the neighborhood of $\mathbf{u}_t^ = \pi_\gamma^*(\mathbf{x}_t)$ which satisfies $\Delta V(\mathbf{x}_t, \mathbf{u}_t^c) < 0$.*

Proof. By Lemma VIII.2, there exists γ_c large enough such that $V_{\pi_\gamma^*}^c(\mathbf{x}_t) < \epsilon < c_{\delta_{\min}}$. Suppose that for every k in the sequence $\gamma_c^k \rightarrow \infty$,

$$\text{dist}(\pi_{\gamma_c^k}^*(\mathbf{x}_t), \mathcal{U}^c(\mathbf{x}_t)) \geq \delta_{\min} \quad \forall k. \quad (45)$$

Therefore,

$$V_{\pi_{\gamma_c^k}^*}^c \geq c(\mathbf{x}_t, \pi_{\gamma_c^k}^*(\mathbf{x}_t)) \geq c_{\delta_{\min}} \quad \forall k, \quad (46)$$

which is a contradiction. Therefore, there exists a large enough γ_c such that $\text{dist}(\pi_{\gamma_c}^*(\mathbf{x}_t), \mathcal{U}^c(\mathbf{x}_t)) < \delta_{\min}$. By Lemma VIII.1, a feasible input $\mathbf{u}_t^c \in \mathcal{U}^c$ exists satisfying $\|\mathbf{u}_t^c - \pi_{\gamma_c}^*(\mathbf{x}_t)\| < \delta$, and thus $\Delta V(\mathbf{x}_t, \mathbf{u}_t^c) < 0$. $\square$

B. Multi-Step Feasible Descent

Next, we show that the prior theorem generalizes when the policy is evaluated over a multi-step feedback policy

$\pi(\mathbf{x}_t, \xi) = (\pi_t(\mathbf{x}_t, \xi), \dots, \pi_{t+N_T-1}(\mathbf{x}_{t+N_T-1}, \xi))$, where $\mathbf{u}_t = \pi_t(\mathbf{x}_t, \xi)$ and $\xi \in \Xi$. First, we define the multi-step feedback dynamics and cost.

Definition VIII.1 (Multi-Step Feedback Dynamics). *The multi-step feedback dynamics is given as $p(\mathbf{x}_{t+N_T} | \mathbf{x}_t, \xi) = \int \dots \int \prod_{i=t}^{t+N_T-1} p(\mathbf{x}_{i+1} | \mathbf{x}_i, \pi_i(\mathbf{x}_i, \xi)) d\mathbf{x}_{t+1} \dots d\mathbf{x}_{t+N_T-1}$.*

Definition VIII.2 (Multi-Step Feedback Cost). *We define*

$$\mathcal{G}_N(\mathbf{x}_t, \xi) = \sum_{i=t}^{t+N-1} \beta^{i-t} g_k(\mathbf{x}_i, \xi) \quad (47)$$

$$g_k(\mathbf{x}_i, \xi) = \int g(\mathbf{x}_i, \pi_i(\mathbf{x}_i, \xi)) p(\mathbf{x}_i | \mathbf{x}_t, \xi) d\mathbf{x}_i \quad (48)$$

as the multi-step feedback cost, with analogous definitions for $\mathcal{L}_N(\mathbf{x}_t, \xi)$, $\ell(\mathbf{x}_t, \xi)$, $\mathcal{C}_N(\mathbf{x}_t, \xi)$, and $c(\mathbf{x}_t, \xi)$.

Given these, we show that there exists a feasible multi-step feedback policy within the neighborhood of an optimal multi-step feedback policy that descends the value function.

Theorem VIII.2. *Under the assumptions of Theorem VIII.1 and a given $\mathbf{x}_t$, for a large enough γ_c , if $\Xi_c^e = \{\xi | \pi_i(\mathbf{x}_i, \xi) \in \mathcal{U}^c(\mathbf{x}_i) \forall i \in \{t, \dots, t+N_T\}, V_{\pi_i(\cdot, \xi)}^c(\mathbf{x}_t) < \epsilon\} \neq \emptyset$, where Ξ is compact, π depends continuously on ξ , and the policy parameterization is rich enough to represent the optimal multi-step feedback policy, $\pi(\cdot, \xi^*) = (\pi_t^*, \dots, \pi_{t+N_T-1}^*)$, then there exist feasible policy parameters ξ_c in the neighborhood of ξ^* that satisfies $c(\mathbf{x}_i, \pi_i(\mathbf{x}_i, \xi_c)) = 0 \forall i \in \{t, \dots, t+N_T-1\}$ and $\Delta V(\mathbf{x}_t, \xi_c) = \beta \mathbb{E}[V(\mathbf{x}_{t+N_T}) | \mathbf{x}_t, \xi_c] - V(\mathbf{x}_t) < 0$.*

Proof. We show that the assumptions necessary for Theorem VIII.1 apply not only to the single-step dynamics and cost, but also to the multi-step versions.

It follows from Definitions VIII.1 and VIII.2 and the assumptions above that the multi-step dynamics, cost, and constraint depend continuously on $\mathbf{x}_t$ and ξ . Likewise, the conditions that $\mathcal{L}_{N_T}(\mathbf{x}_t, \xi) \succ 0$ and $\mathcal{C}_{N_T}(\mathbf{x}_t, \xi) \succeq 0$ follow from definition VIII.2.

Finally, we observe that $V(\mathbf{x}_t)$ is not only the optimal value function for the single-step problem, but also for the multi-step feedback problem when the feedback policy is parametrized by $\xi \in \Xi$. This is shown through repeated application of Equation 11 and the assumption that the policy parameterization is rich enough to represent the optimal multi-step feedback policy:

$$V(\mathbf{x}_t) = \min_{\xi \in \Xi} \left[\mathcal{G}_N(\mathbf{x}_t, \xi) + \beta \mathbb{E}[V(\mathbf{x}_{t+N_T}) | \mathbf{x}_t, \xi] \right]. \quad (49)$$

By applying Theorem VIII.1 to ξ instead of $\mathbf{u}_t$, we show $\exists \xi_c$ s.t. $\Delta V(\mathbf{x}_t, \xi_c) < 0$ and $\mathcal{C}_{N_T}(\mathbf{x}_t, \xi_c) = 0$ which implies $c(\mathbf{x}_i, \pi_i(\mathbf{x}_i, \xi_c)) = 0 \forall i \in \{t, \dots, t+N_T-1\}$. $\square$

Thus, this analysis provides theoretical justification for using NMPC to search for feasible feedback policies in the neighborhood of the optimal policies which descend the value function while strictly adhering to constraints.

C. Approximate Value Functions

The preceding analysis assumes an optimal value function while our proposed algorithm uses learned actor and critic

models. We acknowledge that in practice, these models converge to uncertain approximations of the optimal policy and value function. We define the expected change in the stochastic approximate value function as

$$\Delta \hat{V}(\mathbf{x}_t, \mathbf{u}_t) = \beta \mathbb{E}[V^{\phi\psi}(\mathbf{x}_{t+1}) | \mathbf{x}_t, \mathbf{u}_t] - \mathbb{E}[V^{\phi\psi}(\mathbf{x}_t)]. \quad (50)$$

For a given $\mathbf{x}_t$ and $\mathbf{u}_t$, there is a gap between the change in the approximate and optimal value functions

$$e_V(\mathbf{x}_t, \mathbf{u}_t) = \Delta \hat{V}(\mathbf{x}_t, \mathbf{u}_t) - \Delta V(\mathbf{x}_t, \mathbf{u}_t). \quad (51)$$

To guarantee that the optimal value function, $V(\mathbf{x}_t)$ actually decreases, the expected change in the approximate value function must decrease by more than this gap,

$$\Delta \hat{V}(\mathbf{x}_t, \mathbf{u}_t) < e_V(\mathbf{x}_t, \mathbf{u}_t). \quad (52)$$

Given that $V(\mathbf{x}_t)$ is unknown in practice, this gap is also unknown. Thus, we simply aim to ensure that $\Delta \hat{V}(\mathbf{x}_t, \mathbf{u}_t) < 0$.

D. Simulation Experiment

We explore this analysis empirically using a stochastic Dubins Car dynamical system. The state is given as $\mathbf{x}_t = [r_x \ r_y \ \theta]^T$ where r_x, r_y denote the position and θ is the orientation. The input is given as $\mathbf{u}_t = [v \ \dot{\theta}]^T$ where v is the speed and $\dot{\theta}$ is the angular velocity. The nominal continuous dynamics are given as $\mathbf{f}(\mathbf{x}_t, \mathbf{u}_t) = [v \cos(\theta), v \sin(\theta), \dot{\theta}]^T$. The stochastic, discrete-time dynamics applies Euler integration with $\Delta t = 0.1$ sec and Gaussian process noise with $\Sigma_f = [0.01 \ 0.01 \ 0.01]^T$.

We constructed a simple environment in which the goal is surrounded by dense obstacles arranged on a hexagonal lattice (Fig. 17). Initial states are sampled at positions 7.5m away from the goal with the orientation pointing towards the goal.

We trained 13 actor-critic networks with increasingly large constraint penalties, γ_c . The inputs into these networks were the normalized position, sine/cosine of the orientation, and the normalized range and sine/cosine of the bearing to each obstacle. Each network was trained for 5 million timesteps.

Across 100 trials with differing initial states, we compared the actor policy against AC-PAC-NMPC for each γ_c . We evaluated the percentage of trials in which the system collided with obstacles and reached the goal without colliding. Additionally, we evaluated our RL-based warm-start with and without the actor infeasibility check (lines 6-8 in Algo. 1).

As shown in Figure 18, AC-PAC-NMPC was able to successfully navigate the environment at smaller γ_c values than the actor policy alone. This supports Theorem VIII.2 empirically and shows that for a large enough penalty, a feasible policy exists in the neighborhood of the actor policy which descends the value function. Additionally, these results show that by warm-starting with the prior policy when the actor trajectory is infeasible, the safety of AC-PAC-NMPC can be significantly improved.

IX. CONCLUSION

In this paper, we present an approach for RL-guided SN-MPC to enable long-range navigation in unknown environments while enforcing constraints on probabilistic guarantees of safety. The proposed approach utilized learned actor-critic

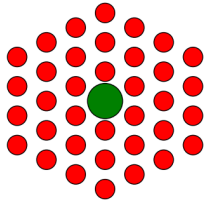


Fig. 17: Green goal, red obstacles.

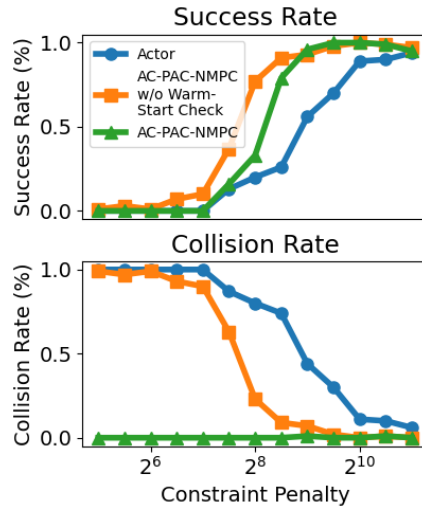


Fig. 18: Success rate and collision rate for each constraint penalty value.

and sensor prediction models to incorporate RL-based warm starts, an uncertainty-aware terminal value function, and a value function improvement constraint into PAC-NMPC. We demonstrated that our approach, both in simulation and on hardware, is capable of navigating a robotic system with high-dimensional, nonlinear, underactuated, stochastic dynamics and local perception through an unknown environment in real-time. We showed that our approach was more robust to model mismatch, sim-to-real transfer, and distribution shift than the RL actor alone. Further, our approach achieved superior long-range performance than PAC-NMPC alone.

There are many interesting directions for future research. While it is beneficial that our approach can utilize actor-critic models trained with standard algorithms, it may be interesting to train it jointly with sensor prediction and PAC-NMPC in the loop. Although Monte Carlo dropout provides a fast approximation of uncertainty in the learned models, our approach would be improved by incorporating a method to calibrate, and potentially bound, the uncertainty estimates. Finally, further analysis into the gap between the estimated value function and the true value function could be explored.

REFERENCES

- [1] D. Q. Mayne, E. C. Kerrigan, E. Van Wyk, and P. Falugi, "Tube-based robust nonlinear model predictive control," *Int. J. Robust Nonlinear control*, vol. 21, pp. 1341–1353, 2011.
- [2] J. Köhler, R. Soloperto, M. A. Müller, and F. Allgöwer, "A computationally efficient robust model predictive control framework for uncertain nonlinear systems," *IEEE Trans. Autom. Control*, vol. 66, pp. 794–801, 2020.
- [3] N. Ozaki, S. Campagnola, and R. Funase, "Tube stochastic optimal control for nonlinear constrained trajectory optimization problems," *J. Guid. Control Dyn.*, vol. 43, pp. 645–655, 2020.
- [4] A. Polevoy, M. Basescu, L. Scheuer, and J. Moore, "Post-stall navigation with fixed-wing uavs using onboard vision," in *2022 Int. Conf. Robot. Automat. (ICRA)*, 2022, pp. 9696–9702.
- [5] D. Falanga, P. Foehn, P. Lu, and D. Scaramuzza, "Pampc: Perception-aware model predictive control for quadrotors," in *2018 IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, pp. 1–8.
- [6] A. Polevoy, M. Gonzales, M. Kobilarov, and J. Moore, "Robust perception-based navigation using pac-nmpc with a learned value function," in *2025 Amer. Control Conf. (ACC)*, pp. 414–421.

- [7] S. Gu, L. Yang, Y. Du, G. Chen, F. Walter, J. Wang, and A. Knoll, "A review of safe reinforcement learning: Methods, theories, and applications," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, pp. 11 216–11 235, 2024.
- [8] L. Brunke, M. Greeff, A. W. Hall, Z. Yuan, S. Zhou, J. Panerati, and A. P. Schoellig, "Safe learning in robotics: From learning-based control to safe reinforcement learning," *Annu. Rev. Control Robot. Auton. Syst.*, vol. 5, pp. 411–444, 2022.
- [9] J. Achiam, D. Held, A. Tamar, and P. Abbeel, "Constrained policy optimization," in *Proc. 34th Int. Conf. Mach. Learn.*, ser. Proc. Mach. Learn. Res., D. Precup and Y. W. Teh, Eds., vol. 70. PMLR, 06–11 Aug 2017, pp. 22–31.
- [10] C. Tessler, D. J. Mankowitz, and S. Mannor, "Reward constrained policy optimization," 2018.
- [11] T. J. Perkins and A. G. Barto, "Lyapunov design for safe reinforcement learning," *J. Mach. Learn. Res.*, vol. 3, pp. 803–832, 2002.
- [12] R. Cheng, G. Orosz, R. M. Murray, and J. W. Burdick, "End-to-end safe reinforcement learning through barrier functions for safety-critical continuous control tasks," in *Proc. AAAI Conf. Artif. Intell.*, vol. 33, no. 01, 2019, pp. 3387–3395.
- [13] T.-H. Pham, G. De Magistris, and R. Tachibana, "Optlayer - practical constrained optimization for deep reinforcement learning in the real world," in *2018 IEEE Int. Conf. Robot. Automat. (ICRA)*, pp. 6236–6243.
- [14] M. Sheckells, G. Garimella, and M. Kobilarov, "Robust policy search with applications to safe vehicle navigation," in *2017 IEEE Int. Conf. Robot. Automat. (ICRA)*, pp. 2343–2349.
- [15] M. Sheckells, G. Garimella, S. Michra, and M. Kobilarov, "Actor-critic pac robust policy search," *ICRA 2019*.
- [16] A. Majumdar, A. Farid, and A. Sonar, "Pac-bayes control: learning policies that provably generalize to novel environments," *Int. J. Robot. Res.*, vol. 40, pp. 574–593, 2021.
- [17] N. Mansard, A. DelPrete, M. Geisert, S. Tonneau, and O. Stasse, "Using a memory of motion to efficiently warm-start a nonlinear predictive controller," in *2018 IEEE Int. Conf. Robot. Automat. (ICRA)*, pp. 2986–2993.
- [18] G. Tang, W. Sun, and K. Hauser, "Learning trajectories for real-time optimal control of quadrotors," in *2018 IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, pp. 3620–3625.
- [19] S. Banerjee, T. Lew, R. Bonalli, A. Alfaadhel, I. A. Alomar, H. M. Shageer, and M. Pavone, "Learning-based warm-starting for fast sequential convex programming and trajectory optimization," in *2020 IEEE Aerosp. Conf.*, pp. 1–8.
- [20] L. Schwenkel, M. Gharbi, S. Trimpe, and C. Ebenbauer, "Online learning with stability guarantees: A memory-based warm starting for real-time mpc," *Automatica*, vol. 122, p. 109247, 2020.
- [21] M. Klaučo, M. Kalúz, and M. Kvasnica, "Machine learning-based warm starting of active set methods in embedded model predictive control," *Eng. Appl. Artif. Intell.*, vol. 77, pp. 1–8, 2019.
- [22] S. W. Chen, T. Wang, N. Atanasov, V. Kumar, and M. Morari, "Large scale model predictive control with neural networks and primal active sets," *Automatica*, vol. 135, p. 109947, 2022.
- [23] J. Briden, C. Choi, K. Yun, R. Linares, and A. Cauligi, "Constraint-informed learning for warm-starting trajectory optimization," *J. Guid. Control Dyn.*, vol. 48, pp. 2272–2287, 2025.
- [24] D. Celestini, D. Gammelli, T. Guffanti, S. D'Amico, E. Capello, and M. Pavone, "Transformer-based model predictive control: Trajectory optimization via sequence modeling," *IEEE Robot. Automat. Lett.*, vol. 9, pp. 9820–9827, 2024.
- [25] M.-K. Bouzidi, Y. Yao, D. Goehring, and J. Reichardt, "Learning-aided warmstart of model predictive control in uncertain fast-changing traffic," in *2024 IEEE Int. Conf. Robot. Automat. (ICRA)*, pp. 14 265–14 271.
- [26] V. Zinage, A. Khalil, and E. Bakolas, "Transformermpc: Accelerating model predictive control via transformers," in *2025 IEEE Int. Conf. Robot. Automat. (ICRA)*, pp. 9221–9227.
- [27] A. Haffemayer, A. Chapin, A. Jordana, K. Wojciechowski, F. Lamiraux, N. Mansard, and V. Petrik, "Warm-starting collision-free model predictive control with object-centric diffusion," *IEEE Robot. Automat. Lett.*, vol. 11, pp. 4417–4424, 2026.
- [28] T. S. Lembono, C. Mastalli, P. Fernbach, N. Mansard, and S. Calinon, "Learning how to walk: Warm-starting optimal control solver with memory of motion," in *2020 IEEE Int. Conf. Robot. Automat. (ICRA)*, 2020, pp. 1357–1363.
- [29] S. Banerjee, A. Cauligi, and M. Pavone, "Deep learning warm starts for trajectory optimization on the international space station," in *2025 Int. Conf. Space Robot. (iSpaRo)*, pp. 394–401.

- [30] S. Sharma and M. E. Taylor, "Autonomous waypoint generation strategy for on-line navigation in unknown environments," *environment*, vol. 2, p. 3D, 2012.
- [31] C. Greatwood and A. G. Richards, "Reinforcement learning and model predictive control for robust embedded quadrotor guidance and control," *Auton. Robots*, vol. 43, pp. 1681–1693, 2019.
- [32] B. Brito, M. Everett, J. P. How, and J. Alonso-Mora, "Where to go next: Learning a subgoal recommendation policy for navigation in dynamic environments," *IEEE Robot. Automat. Lett.*, vol. 6, pp. 4616–4623, 2021.
- [33] E. Kaufmann, A. Loquercio, R. Ranftl, A. Dosovitskiy, V. Koltun, and D. Scaramuzza, "Deep drone racing: Learning agile flight in dynamic environments," in *Conf. Robot Learn.* PMLR, 2018, pp. 133–145.
- [34] S. Bansal, V. Tolani, S. Gupta, J. Malik, and C. Tomlin, "Combining optimal control and learning for visual navigation in novel environments," in *Conf. Robot Learn.* PMLR, 2020, pp. 420–429.
- [35] K. Chua, R. Calandra, R. McAllister, and S. Levine, "Deep reinforcement learning in a handful of trials using probabilistic dynamics models," in *Advances Neural Inf. Process. Syst.*, S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, Eds., vol. 31. Curran Associates, Inc., 2018.
- [36] K. Lowrey, A. Rajeswaran, S. Kakade, E. Todorov, and I. Mordatch, "Plan online, learn offline: Efficient learning and exploration via model-based control," in *Int. Conf. Learn. Representations*, 2019.
- [37] M. Bhardwaj, A. Handa, D. Fox, and B. Boots, "Information theoretic model predictive q-learning," in *Proc. 2nd Conf. on Learn. Dyn. Control*, ser. Proc. Mach. Learn. Res., A. M. Bayen, A. Jadbabaie, G. Pappas, P. A. Parrilo, B. Recht, C. Tomlin, and M. Zeilinger, Eds., vol. 120. PMLR, 10–11 Jun 2020, pp. 840–850.
- [38] D. Hoeller, F. Farshidian, and M. Hutter, "Deep value model predictive control," in *Proc. Conf. Robot Learn.*, ser. Proc. Mach. Learn. Res., L. P. Kaelbling, D. Kragic, and K. Sugiura, Eds., vol. 100. PMLR, 30 Oct–01 Nov 2020, pp. 990–1004.
- [39] G. Grandesso, E. Alboni, G. P. R. Papini, P. M. Wensing, and A. D. Prete, "Cacto: Continuous actor-critic with trajectory optimization-towards global optimality," *IEEE Robot. Automat. Lett.*, vol. 8, pp. 3318–3325, 2023.
- [40] M. Zanon and S. Gros, "Safe reinforcement learning using robust mpc," *IEEE Trans. Autom. Control*, vol. 66, pp. 3638–3652, 2021.
- [41] K. P. Wabersich and M. N. Zeilinger, "A predictive safety filter for learning-based control of constrained nonlinear dynamical systems," *Automatica*, vol. 129, p. 109597, 2021.
- [42] H. Sikchi, W. Zhou, and D. Held, "Learning off-policy with online planning," in *Proc. 5th Conf. Robot Learn.*, ser. Proc. Mach. Learn. Res., A. Faust, D. Hsu, and G. Neumann, Eds., vol. 164. PMLR, 08–11 Nov 2022, pp. 1622–1633.
- [43] N. A. Hansen, H. Su, and X. Wang, "Temporal difference learning for model predictive control," in *Proc. 39th Int. Conf. Mach. Learn.*, ser. Proc. Mach. Learn. Res., K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvari, G. Niu, and S. Sabato, Eds., vol. 162. PMLR, 17–23 Jul 2022, pp. 8387–8406.
- [44] P. Karkus, B. Ivanovic, S. Mannor, and M. Pavone, "Diffstack: A differentiable and modular control stack for autonomous vehicles," in *Proc. 6th Conf. Robot Learn.*, ser. Proc. Mach. Learn. Res., K. Liu, D. Kulic, and J. Ichnowski, Eds., vol. 205. PMLR, 14–18 Dec 2023, pp. 2170–2180.
- [45] A. Romero, Y. Song, and D. Scaramuzza, "Actor-critic model predictive control," in *2024 IEEE Int. Conf. Robot. Automat. (ICRA)*, pp. 14 777–14 784.
- [46] A. Romero, E. Aljalbout, Y. Song, and D. Scaramuzza, "Actor-critic model predictive control: Differentiable optimization meets reinforcement learning for agile flight," *IEEE Trans. Robot.*, vol. 42, pp. 673–692, 2026.
- [47] J. Moore, R. Cory, and R. Tedrake, "Robust post-stall perching with a simple fixed-wing glider using lqr-trees," *Bioinspiration & biomimetics*, vol. 9, p. 025013, 2014.
- [48] A. Majumdar and R. Tedrake, "Funnel libraries for real-time robust feedback motion planning," *Int. J. Robot. Res.*, vol. 36, pp. 947–982, 2017.
- [49] A. J. Barry, P. R. Florence, and R. Tedrake, "High-speed autonomous obstacle avoidance with pushbroom stereo," *J. Field Robot.*, vol. 35, pp. 52–68, 2018.
- [50] J. M. Levin, A. Paranjape, and M. Nahon, "Agile fixed-wing uav motion planning with knife-edge maneuvers," in *2017 Int. Conf. Unmanned Aircr. Syst. (ICUAS)*. IEEE, pp. 114–123.
- [51] J. M. Levin, M. Nahon, and A. A. Paranjape, "Real-time motion planning with a fixed-wing uav using an agile maneuver space," *Auton. Robots*, vol. 43, pp. 2111–2130, 2019.
- [52] E. Bulka and M. Nahon, "High-speed obstacle-avoidance with agile fixed-wing aircraft," in *2019 Int. Conf. Unmanned Aircr. Syst. (ICUAS)*. IEEE, pp. 971–980.
- [53] H. Alturbeh and J. F. Whidborne, "Real-time obstacle collision avoidance for fixed wing aircraft using b-splines," in *2014 UKACC Int. Conf. Control (CONTROL)*, pp. 115–120.
- [54] A. Bry, C. Richter, A. Bachrach, and N. Roy, "Aggressive flight of fixed-wing and quadrotor aircraft in dense indoor environments," *Int. J. Robot. Res.*, vol. 34, pp. 969–1002, 2015.
- [55] L. Morando, S. A. Salunkhe, N. Bobbili, J. Mao, L. Masci, C. de Souza, N. Hung, and G. Loianno, "Trajectory planning and control for differentially flat fixed-wing aerial systems," in *2025 IEEE Int. Conf. Robot. Automat. (ICRA)*. IEEE, pp. 6336–6342.
- [56] M. Basescu and J. Moore, "Direct nmpe for post-stall motion planning with fixed-wing uavs," in *2020 IEEE Int. Conf. Robot. Automat. (ICRA)*, pp. 9592–9598.
- [57] M. Basescu, A. Polevoy, B. Yeh, L. Scheuer, E. Sutton, and J. Moore, "Agile fixed-wing uavs for urban swarm operations," *IEEE Trans. Field Robot.*, vol. 1, pp. 394–423, 2024.
- [58] V. Madabushi, Y. Kopel, A. Polevoy, and J. Moore, "Dense fixed-wing swarming using receding-horizon nmpe," in *2025 IEEE Int. Conf. Robot. Automat. (ICRA)*, pp. 8656–8662.
- [59] J. Pravitra, E. Theodorou, and E. N. Johnson, "Flying complex maneuvers with model predictive path integral control," in *AIAA Scitech 2021 forum*, 2021, p. 1957.
- [60] A. Polevoy, M. Kobilarov, and J. Moore, "Probably approximately correct nonlinear model predictive control (pac-nmpe)," *IEEE Robot. Automat. Lett.*, vol. 8, pp. 7226–7233, 2023.
- [61] E. Tal, G. Ryou, and S. Karaman, "Aerobatic trajectory generation for a vtol fixed-wing aircraft using differential flatness," *IEEE Trans. Robot.*, vol. 39, pp. 4805–4819, 2023.
- [62] E. Bulka and M. Nahon, "Reactive obstacle-avoidance for agile, fixed-wing, unmanned aerial vehicles," *Field Robot.*, vol. 2, pp. 1507–1566, 2022.
- [63] R. Tedrake, *Underactuated Robotics*, 2022.
- [64] R. Sutton and A. Barto, *Reinforcement learning: An introduction*, 2020.
- [65] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *preprint arXiv:1707.06347*, 2017.
- [66] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel *et al.*, "Soft actor-critic algorithms and applications," *preprint arXiv:1812.05905*, 2018.
- [67] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *Proc. 35th Int. Conf. Mach. Learn.*, ser. Proc. Mach. Learn. Res., J. Dy and A. Krause, Eds., vol. 80. PMLR, 10–15 Jul 2018, pp. 1587–1596.
- [68] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: a simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, pp. 1929–1958, 2014.
- [69] Y. Gal and Z. Ghahramani, "Dropout as a bayesian approximation: Representing model uncertainty in deep learning," in *Proc. 33rd Int. Conf. Mach. Learn.*, ser. Proc. Mach. Learn. Res., M. F. Balcan and K. Q. Weinberger, Eds., vol. 48. New York, New York, USA: PMLR, 20–22 Jun 2016, pp. 1050–1059.
- [70] P. E. Gill, W. Murray, and M. A. Saunders, "Snopt: An sqp algorithm for large-scale constrained optimization," *SIAM review*, vol. 47, pp. 99–131, 2005.
- [71] S. Huang, R. F. J. Dossa, C. Ye, J. Braga, D. Chakraborty, K. Mehta, and J. G. Araújo, "Cleanrl: High-quality single-file implementations of deep reinforcement learning algorithms," *J. Mach. Learn. Res.*, vol. 23, pp. 1–18, 2022.
- [72] S. Macenski, F. Martin, R. White, and J. Ginés Clavero, "The marathon 2: A navigation system," in *2020 IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*.
- [73] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou, "Aggressive driving with model predictive path integral control," in *2016 IEEE Int. Conf. Robot. Automat. (ICRA)*, pp. 1433–1440.
- [74] C. Rucker, "Integrating rotations using nonunit quaternions," *IEEE Robot. Automat. Lett.*, vol. 3, pp. 2979–2986, 2018.
- [75] S. F. Hoerner and H. V. Borst, "Fluid-dynamic lift: practical information on aerodynamic and hydrodynamic lift," 1985.
- [76] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *Med. Image Compute. Comput.-Assist. Interv. (MICCAI) 2015*, N. Navab, J. Hornegger, W. M. Wells, and A. F. Frangi, Eds. Cham: Springer Int. Publishing, pp. 234–241.

- [77] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: from error visibility to structural similarity," *IEEE Trans. Image Process.*, vol. 13, pp. 600–612, 2004.
- [78] E. Perez, F. Strub, H. De Vries, V. Dumoulin, and A. Courville, "Film: Visual reasoning with a general conditioning layer," in *Proc. AAAI Conf. Artif. Intell.*, vol. 32, no. 1, 2018.